```
# ===== edge/detector/world_detector.py =====
"""开放词汇检测器（YOLO-World 通道）—— 零样本，不需要一张标注图。
它在本项目里的定位（先读这一段，否则会误用）
----------------------------------------------
本通道不是主检测器，也不是"更准的 OpenCV"。它是：
- 能力演示：现场说一个新词（"塑料瓶""渔网""泡沫浮球"），
  系统立刻能检出这一类，不需要重新采集、标注、训练；
- 对照评测：与 CvDetector 跑同一批图，输出定性对照（检出情况 + 时延），
  默认不进告警主链路（edge/config.yaml 的 detector.backend 默认 cv）。
它不承诺精度。原因见下面的标定依据：这是本模块存在的全部诚实性所在。
标定依据（本仓库历史实测，不是估计）
-------------------------------------
artifacts/marine-detections-yoloworld.json（imgsz=1600, conf=0.02,
yolov8s-worldv2.pt）在 5 张演示原图上的结果：
==========================  ====  ========  ==========
图                          检出  最高置信  ≥0.45 的条数
==========================  ====  ========  ==========
beach.jpg                      8     0.123           0
foam.jpg                       3     0.048           0
midway.jpg                     7     0.413           0
net.jpg                        7     0.107           0
plastic.jpg                   16     0.294           0
==========================  ====  ========  ==========
artifacts/vision-preview/ 下四份 prompt 标定记录里，
历史最高置信度为 0.184（foam cup），标定阈值被压到 0.004~0.012 才拿到框。
结论有两层，都很重要：
1. 绝不能拿 0.45 当本通道的门限。 0.45 是 TemporalValidator 的
   *时序*门限，用来过滤 CvDetector 的伪置信度；开放词汇模型的输出分数
   是另一个尺度（中位数 0.04 量级）。把两者混为一谈，本通道的输出必然是空的。
   所以本模块有自己的 conf / temporal_min_confidence（默认 0.02，
   取值依据见 DEFAULT_WORLD_CONFIG 的注释与 docs/decisions.md ADR-026）。
2. 姿态要放低。 在当前可得的图上，它是"能指出候选"的水平，不是"能报精度"的水平。
   对外只讲"零样本、可现场加词"，不讲 precision / recall。
输出契约（与 CvDetector 完全一致，下游一行都不用改）
------------------------------------------------------
   [{"class": "foam", "confidence": 0.72, "bbox": [x1, y1, x2, y2]}, ...]
class 一定是 foam / plastic / fishing_gear / other 之一
（由 label_map 把提示词原文映射过来），bbox 是设备原始分辨率下的
像素坐标、原点左上。
可选依赖的处理
--------------
torch / ultralytics / transformers 不在边缘测试环境里，也不该被强行拖进来。
所以：
- 顶层不导入任何重依赖；
- 模型懒加载（第一次 detect 才建）；
- 依赖或权重缺失时 available 为 False、last_error 给出可执行提示、
  detect() 返回空列表并把这一帧计入 stats()["unavailable_calls"]
  ——不抛异常拖垮主循环，也不静默假装"这一帧没有垃圾"；
- 测试通过注入 runner 绕开真实模型，因此契约测试不需要 torch。
"""
```

```
from __future__ import annotations


import math
from pathlib import Path
from typing import Any, Callable
import cv2
import numpy as np
from label_map import (
    CONTRACT_CLASSES,
    DEMO_DEFAULT_CLASSES,
    map_prompt,
    unmapped_prompts,
  )
BACKEND_NAME = "world"
# 仓库根目录（本文件在 <root>/edge/detector/world_detector.py）。
#   用来把配置里的相对权重路径解析成确定的位置。
  #
#   为什么必须这么做（这是实测踩到的坑，且后果很严重）：
#   `config.yaml` 里写的是裸文件名 `yolov8s-worldv2.pt`，而 ultralytics 是
#   按当前工作目录找权重的。于是：
#     - 从仓库根跑 → 找到本地文件，正常启动；
#     - 从 `edge/` 跑（`edge/main.py` 的常规用法）→ 找不到 →
#       ultralytics 静默转向网络下载，连试 3 次，每次都在
#       `curl: (56) schannel: server closed abruptly` 上卡住，
#       整个启动挂住 2 分钟以上且没有任何异常。
#   现场演示正是"从 edge/ 启动"，所以这条会把演示直接卡死，
#   而且现象只是"程序没反应"，看不出是在下载权重。
PROJECT_ROOT = Path(__file__).resolve().parents[2]
def resolve_weights_path(raw: str | Path) -> Path:
    """把配置里的权重路径解析成绝对路径（相对路径按仓库根解析，不按 cwd）。
    裸文件名（如 yolov8s-worldv2.pt）在从仓库根启动时恰好可用，
    所以这个缺陷平时不会暴露：只有换一个 cwd 才现形（详见 PROJECT_ROOT 注释）。
    """
    path = Path(str(raw)).expanduser()
    return path if path.is_absolute() else (PROJECT_ROOT / path)
# 原始检测的中间形态（runner 的返回约定）：
#   {"prompt": "plastic bottle", "confidence": 0.21, "bbox": [x1, y1, x2, y2]}
RawDetection = dict[str, Any]
# 注入式 runner：(frame, prompts, params) -> [RawDetection, ...]
Runner = Callable[[np.ndarray, list[str], dict[str, Any]], list[RawDetection]]
# ---------------------------------------------------------------------------
# 参数默认值
# 与 edge/config.yaml 的 detector.world 段逐项对账（test_world_detector.py 里有断言）。
#   刻意不复用 detector.py 的 DEFAULT_CONFIG：那是 OpenCV 通道的参数集，
#   两者的置信度尺度不同，混在一个字典里早晚会被谁改错。
# ---------------------------------------------------------------------------
DEFAULT_WORLD_CONFIG: dict[str, Any] = {
    # 权重与文本编码器
    "weights": "yolov8s-worldv2.pt",
    "clip_dir": "~/.cache/clip-vit-base-patch32",
     # 推理参数：CPU 是刻意的默认值（本机 GPU 与现有 torch 构建不匹配，见 README）
```

```
    "device": "cpu",



    "imgsz": 960,
    # 模型侧打分门限。
    #   不是 0.45：那是 cv 通道的时序门限，两套尺度不同（ADR-024）。
    #
    #   为什么是 0.02：本通道的用途是"现场加一个词就能检出这一类"，
    #   所以默认门限必须让四个契约类别都可报，否则演示会出现
    #   "加词没反应"，而实际原因只是门限把它滤掉了。
    #   实测（artifacts/metrics/vision_backends_latest.json 的 threshold_probe，
    #   5 张 4K 真实照片、门限压到 0.001）逐类最高分：
    #       other 0.5824 / fishing_gear 0.0514 / plastic 0.0355 / foam 0.0260
    #   → foam 是最紧的一类，门限必须低于 0.026 才能让它可报，故取 0.02。
    #
    #   这是为满足"四类皆可报"这一功能性要求选的演示工作点，
    #     不是精度最优门限：独立测试集真值标注为 0，没有任何数据能标定它。
    #     严禁把 0.02 写成"最优"或"经标定"。现场如需换门限，
    #     用 `--world-conf` 或改 edge/config.yaml，两者都即时生效。
    "conf": 0.02,
    "iou": 0.45,
    "max_det": 60,
    # 喂给 TemporalValidator 的门限。与 conf 同尺度——
    #   若沿用 cv 通道的 0.45，本通道的检测会在时序环节被全部丢掉。
    "temporal_min_confidence": 0.02,
    # 默认提示词：取自演示脚本的历史默认集（label_map.DEMO_DEFAULT_CLASSES 有对账测试）
    "prompt_classes": list(DEMO_DEFAULT_CLASSES),
    # 是否允许 ultralytics 在本地找不到权重时自动联网下载。
    #   默认 False：现场网络不可预期，静默下载会让启动时间不可控
    #    （实测：联网重试 3 次、卡住 2 分多钟、最后仍可能失败，且全程无异常抛出）。
    #   部署/现场需要首次下载时，显式把 edge/config.yaml 里这一项改成 true，
    #   或直接按 resolve_weights_path 解析出的路径手工放好权重文件。
    "allow_weights_download": False,
}
def _cfg(config: dict[str, Any] | None, key: str) -> Any:
    """逐键取值，缺失回落到默认。
    逐键而不是整体 `config or DEFAULT_WORLD_CONFIG`：
    现场边缘盒上的 config.yaml 很可能是旧版（少几个键），
    整体回退会让运维在 yaml 里改的参数静默失效：本项目反复出现的缺陷形状。
    """
    if config and key in config:
        return config[key]
    return DEFAULT_WORLD_CONFIG[key]
class WorldDetector:
    """开放词汇检测器。接口与 CvDetector 对齐（detect / reset）。
    用法（边缘主程序）：
        det = WorldDetector(config["world"], roi=cfg.get("roi"))
        if not det.available:
            print(det.last_error)   # 把"缺什么"说清楚，不要让它表现成"检测不到"
        for frame in stream:
            for d in det.detect(frame):
                ...  # 契约格式，可直接送时序校验
```

现场加词（演示时的核心动作）：

```
    det.set_prompts(["plastic bottle", "fishing net", "foam buoy"])
测试时注入 runner，不依赖 torch：
    WorldDetector(cfg, runner=lambda frame, prompts, params: [...])
"""
backend_name = BACKEND_NAME
def __init__(
    self,
    config: dict[str, Any] | None = None,
    roi: list[list[float]] | None = None,
    runner: Runner | None = None,
) -> None:
    self.config = config or {}
    # 注入的 runner（测试夹具）：一旦注入就固定用它，set_prompts 不会作废它
    self._injected_runner: Runner | None = runner
    # 懒建的真实 runner 与其模型句柄（set_prompts 时对模型直接换词表，不重建）
    self._model_runner: Runner | None = None
    self._model: Any | None = None
    self._runner_error: str | None = None
    self._env_probe: tuple[bool, str] | None = None
    self._roi = roi or []
    self._roi_poly: np.ndarray | None = None
    self._roi_shape: tuple[int, int] | None = None
    prompts = _cfg(config, "prompt_classes") or []
    self._prompts: list[str] = [str(p) for p in prompts]
    # 提示词 → (契约类别，match_kind) 的缓存，set_prompts 时重建
    self._label_cache: dict[str, tuple[str, str]] = {}
    self._rebuild_label_cache()
    self._stats: dict[str, int] = {
        "calls": 0,
        "raw": 0,
        "kept": 0,
        "dropped_low_conf": 0,
        "dropped_roi": 0,
        "dropped_bad_box": 0,
        "unavailable_calls": 0,
        "fallback_other": 0,
    }
# ------------------------------------------------------------------
# 加载（把"能不能跑"的判定提前到启动阶段）
# ------------------------------------------------------------------
def load(self) -> bool:
    """强制构建模型一次，返回是否成功。
    为什么 `available` 不够
    -------------------------
    available 只做依赖与权重的存在性探测，不加载模型（要便宜、可在日志里随便调）。
    它因此发现不了下面这一类失败：依赖齐、权重在，但模型建不起来。
    真实案例（2026-09-27 实测）：`torch 2.5.1` + `ultralytics 8.4.35`，
    三项探测全绿，但 ultralytics 因 CVE-2025-32434 拒绝在 torch<2.6 上
    加载 .pt（报错原文：*require users to upgrade torch to at least v2.6*）。
```

```
        于是 `available == True`、每帧都返回 0 检出。



        只问 `available` 就会把"这一路根本没跑起来"当成"画面里没有垃圾"——
        两者在数字上一模一样。所以主程序启动时必须调本方法。
        失败原因会写进 last_error，可用性相关的统计也随之更新。
        """
        if self._ensure_runner() is not None:
            return True
        if self._runner_error is None:
            self._runner_error = "模型未能构建（未知原因）"
        return False
    # ------------------------------------------------------------------
    # 状态自述
    # ------------------------------------------------------------------
    @property
    def prompts(self) -> tuple[str, ...]:
        """当前生效的提示词（现场加词后即是新的一套）。"""
        return tuple(self._prompts)
    @property
    def last_error(self) -> str | None:
        """最近一次不可用的原因；可用时为 None。"""
        return self._runner_error
    @property
    def available(self) -> bool:
        """本通道当前能否真正推理。
        惰性判定：注入 runner（测试）时为 True；否则只做依赖与权重的
        存在性探测，不加载模型，因此可以安全地放在启动日志里。
        探测失败时会把原因写进 last_error：否则主程序的失败提示
        只能打出"不可用：None"，等于没说。（这是实现时踩到过的坑：
        `available` 探到了原因却没存下来。）
        """
        if self._injected_runner is not None:
            return True
        ok, reason = self._probe_environment()
        if not ok and self._runner_error is None:
            self._runner_error = reason
        return ok
    @property
    def temporal_min_confidence(self) -> float:
        """本通道应当使用的时序门限。
        这是原方案最容易出错的一处：把 cv 通道的 0.45 套到本通道上，
        检测会在时序环节被全部丢掉，而现象只是"world 通道没结果"。
        主程序请从这里取值，不要在别处再写一遍配置键。
        """
        return float(_cfg(self.config, "temporal_min_confidence"))
    def stats(self) -> dict[str, Any]:
        """把"为什么少了检测框"拆成可解释的计数。
        没有这组计数，fallback_other（提示词没归宿）与
        dropped_low_conf（门限太高）在现象上一模一样：某个类别恒为 0。
        """
        out: dict[str, Any] = dict(self._stats)
```

```
        out["prompts"] = list(self._prompts)
        out["unmapped_prompts"] = unmapped_prompts(self._prompts)
        out["conf"] = float(_cfg(self.config, "conf"))
        out["temporal_min_confidence"] = self.temporal_min_confidence
        out["available"] = self.available
        out["last_error"] = self._runner_error
        return out
    # ------------------------------------------------------------------
    # 提示词（现场加词）
    # ------------------------------------------------------------------
    def set_prompts(self, prompts: list[str] | tuple[str, ...]) -> None:
        """替换提示词集，下一次 detect 生效。
        现场演示就是这一句：评委说一个词，加进来，立刻能检出这一类。
        不重训、不标注、不改代码：这是本通道相对 CvDetector 的
        唯一硬性优势，也是它值得存在的理由。
        提示词会经 label_map 映射成四类；没有归宿的词回落 other
        并计入 stats()["fallback_other"]，不静默丢弃。
        """
        new_prompts = [str(p) for p in prompts]
        if not new_prompts:
            raise ValueError("提示词不能为空：开放词汇检测器至少需要一个提示词")
        self._prompts = new_prompts
        self._rebuild_label_cache()
        # 已建模则直接对模型换词表（比重新加载权重快得多）；
        # 注入式 runner 由测试夹具自行决定语义，这里不动它。
        if self._model is not None:
            self._model.set_classes(list(self._prompts))
    def _rebuild_label_cache(self) -> None:
        self._label_cache = {p: map_prompt(p) for p in self._prompts}
    # ------------------------------------------------------------------
    # 主入口
    # ------------------------------------------------------------------
    def detect(self, frame: np.ndarray) -> list[dict[str, Any]]:
        """对一帧做检测，返回契约格式的检测列表。
        不可用时返回 [] 并把该次计入 unavailable_calls。
        这样主循环不会因为缺依赖而崩，但统计里能立刻看出"这一路根本没跑"。
        """
        if frame is None or frame.size == 0:
            return []
        self._stats["calls"] += 1
        runner = self._ensure_runner()
        if runner is None:
            self._stats["unavailable_calls"] += 1
            return []
        h, w = frame.shape[:2]
        self._sync_roi((h, w))
        params = {
            "imgsz": int(_cfg(self.config, "imgsz")),
            "conf": float(_cfg(self.config, "conf")),
            "iou": float(_cfg(self.config, "iou")),
```

```
            "max_det": int(_cfg(self.config, "max_det")),
            "device": str(_cfg(self.config, "device")),
        }
        try:
            raw = runner(frame, list(self._prompts), params)
        except Exception as exc:  # noqa: BLE001：单帧失败不该拖垮取流循环
            self._runner_error = f"本帧推理失败：{exc}"
            self._stats["unavailable_calls"] += 1
            return []
        self._stats["raw"] += len(raw)
        results: list[dict[str, Any]] = []
        for item in raw:
            det = self._build(item, w, h)
            if det is not None:
                results.append(det)
        # 稳定排序：置信度降序，同分按 bbox 字典序：同一输入两次跑结果一致
        results.sort(key=lambda d: (-d["confidence"], d["bbox"]))
        self._stats["kept"] += len(results)
        return results
    def reset(self) -> None:
        """重置（与 CvDetector.reset 对齐）。
        本通道是无状态的（不像背景建模那样有历史），
        所以这里只清统计计数，不动模型与提示词。
        """
        for key in self._stats:
            self._stats[key] = 0
    # ------------------------------------------------------------------
    # 单条原始检测 → 契约
    # ------------------------------------------------------------------
    def _build(self, item: RawDetection, w: int, h: int) -> dict[str, Any] | None:
        prompt = str(item.get("prompt", ""))
        numeric = self._extract_numbers(item)
        if numeric is None:
            return None
        conf, x1, y1, x2, y2 = numeric
        if conf < float(_cfg(self.config, "conf")):
            self._stats["dropped_low_conf"] += 1
            return None
        if x2 <= x1 or y2 <= y1:
            self._stats["dropped_bad_box"] += 1
            return None
        # 裁剪到画面内（模型偶发越界框；下游按坐标做网格归并，越界会污染热力图）
        bx1 = _clamp_int(x1, 0, w)
        by1 = _clamp_int(y1, 0, h)
        bx2 = _clamp_int(x2, 0, w)
        by2 = _clamp_int(y2, 0, h)
        if bx2 <= bx1 or by2 <= by1:
            self._stats["dropped_bad_box"] += 1
            return None
        if not self._keep_by_roi(bx1, by1, bx2, by2):
```

```
            self._stats["dropped_roi"] += 1
            return None
        # 类别映射放在最后：只有真要输出的框才计入 fallback_other，
        # 否则"被门限丢掉的框"也会把提示词问题的计数抬高，读数就没法用了。
        cls, kind = self._label_for(prompt)
        if kind != "exact":
            self._stats["fallback_other"] += 1
        return {"class": cls, "confidence": round(conf, 4), "bbox": [bx1, by1, bx2, by2]}
    def _extract_numbers(
        self, item: RawDetection
    ) -> tuple[float, float, float, float, float] | None:
        """取出并校验 (confidence, x1, y1, x2, y2)；缺失/非数/非有限值返回 None。"""
        try:
            conf = float(item.get("confidence", 0.0))
            bbox = item.get("bbox", ())
            x1, y1, x2, y2 = (float(v) for v in bbox)
        except (TypeError, ValueError):
            self._stats["dropped_bad_box"] += 1
            return None
        for value in (conf, x1, y1, x2, y2):
            if not math.isfinite(value):
                self._stats["dropped_bad_box"] += 1
                return None
        return conf, x1, y1, x2, y2
    def _label_for(self, prompt: str) -> tuple[str, str]:
        match = self._label_cache.get(prompt)
        if match is None:
            # 模型给出的 prompt 不在当前词表里（例如词表刚被外部改过）
            match = map_prompt(prompt)
            self._label_cache[prompt] = match
        return match
    # ------------------------------------------------------------------
    # ROI
    # ------------------------------------------------------------------
    def _sync_roi(self, shape: tuple[int, int]) -> None:
        """按当前帧尺寸构建 ROI 多边形（相对坐标 → 像素），尺寸变了就重建。"""
        if not self._roi:
            self._roi_poly = None
            self._roi_shape = None
            return
        if self._roi_poly is not None and self._roi_shape == shape:
            return
        h, w = shape
        self._roi_poly = np.array(
            [[float(x) * w, float(y) * h] for x, y in self._roi], dtype=np.float32
        )
        self._roi_shape = shape
    def _keep_by_roi(self, x1: int, y1: int, x2: int, y2: int) -> bool:
        """框中心是否落在 ROI 多边形内。
        与 CvDetector 的 ROI 语义刻意一致（相对坐标多边形、空表示全画面），
```

```
401         但判定方式不同：那边是在掩膜上逐像素裁剪，这边是按框中心判断。
402
403
404         所以换通道时 ROI 的边界行为会有细微差别：这条应当写进部署说明，
405         而不是让运维在现场自己发现。
406         """
407         if self._roi_poly is None:
408             return True
409         cx = (x1 + x2) / 2.0
410         cy = (y1 + y2) / 2.0
411         return cv2.pointPolygonTest(self._roi_poly, (cx, cy), False) >= 0
412     # ------------------------------------------------------------------
413     # runner：真实模型 / 注入
414     # ------------------------------------------------------------------
415     def _probe_environment(self) -> tuple[bool, str]:
416         """探测依赖与权重是否就绪（不加载模型），结果缓存一次。"""
417         if self._env_probe is not None:
418             return self._env_probe
419         try:
420             from clip_shim import (  # noqa: PLC0415
421                 clip_available,
422                 clip_dir_status,
423                 ultralytics_available,
424             )
425         except ImportError as exc:  # pragma: no cover - 同目录模块，正常不会缺
426             self._env_probe = (False, f"clip_shim 不可导入：{exc}")
427             return self._env_probe
428         deps_ok, deps_reason = clip_available()
429         if not deps_ok:
430             self._env_probe = (False, deps_reason)
431             return self._env_probe
432         clip_dir = Path(str(_cfg(self.config, "clip_dir"))).expanduser()
433         weights_ok, weights_reason = clip_dir_status(clip_dir)
434         if not weights_ok:
435             self._env_probe = (False, weights_reason)
436             return self._env_probe
437         # 检测权重也要在这一步探到。只探 CLIP 目录会漏掉"权重不在"，
438         #   而那正是"从 edge/ 启动时静默联网下载"的入口（见 PROJECT_ROOT 注释）。
439         #   探到就能在启动阶段给出准确原因，而不是等到 load() 才失败。
440         det_weights = resolve_weights_path(_cfg(self.config, "weights"))
441         if not det_weights.exists() and not bool(
442             _cfg(self.config, "allow_weights_download")
443         ):
444             self._env_probe = (
445                 False,
446                 f"权重文件不存在：{det_weights}"
447                 "（相对路径按仓库根解析，不按当前工作目录；"
448                 "如需联网下载请置 detector.world.allow_weights_download=true）",
449             )
450             return self._env_probe
451         ultra_ok, ultra_reason = ultralytics_available()
452         if not ultra_ok:
```

```
            self._env_probe = (False, ultra_reason)



            return self._env_probe
        self._env_probe = (True, f"就绪 ({deps_reason}; {ultra_reason}) ")
        return self._env_probe
    def _ensure_runner(self) -> Runner | None:
        if self._injected_runner is not None:
            return self._injected_runner
        if self._model_runner is not None:
            return self._model_runner
        if self._runner_error is not None:
            return None  # 已经失败过，不重复尝试（避免每帧都去 import 并打日志）
        ok, reason = self._probe_environment()
        if not ok:
            self._runner_error = reason
            return None
        try:
            self._model_runner = self._build_ultralytics_runner()
        except Exception as exc:  # noqa: BLE001
            self._runner_error = f"模型加载失败：{exc}"
            return None
        return self._model_runner
    def _build_ultralytics_runner(self) -> Runner:
        """构造真实 runner：装 CLIP 垫片 → 建 YOLO-World → 子类词表。
        只在依赖齐备时被调用。
        """
        from clip_shim import install_clip_shim  # noqa: PLC0415
        clip_dir = Path(str(_cfg(self.config, "clip_dir"))).expanduser()
        install_clip_shim(clip_dir)
        # 权重路径必须在这里解析成绝对路径后再交给 ultralytics。
        #   传裸文件名等于把"用哪个权重"这件事交给 cwd 决定（详见 PROJECT_ROOT 注释），
        #   从 edge/ 启动会退化成联网下载并把启动卡住数分钟。
        weights_path = resolve_weights_path(_cfg(self.config, "weights"))
        if not weights_path.exists():
            if not bool(_cfg(self.config, "allow_weights_download")):
                # 明确失败，而不是让 ultralytics 静默下载：
                # 现场没有网络时，"下载中"和"卡死"在观测上无法区分。
                raise FileNotFoundError(
                    f"权重文件不存在：{weights_path}。"
                    "（相对路径按仓库根解析，不按当前工作目录）"
                    "请把权重放到该路径，或把 edge/config.yaml 的 "
                    "detector.world.allow_weights_download 设为 true 允许联网下载。"
                )
        from ultralytics import YOLOWorld  # noqa: PLC0415
        # 本地存在时必须传字符串形式的绝对路径：
        # ultralytics 见到绝对路径就不会再去解析/下载。
        model = YOLOWorld(str(weights_path))
        model.set_classes(list(self._prompts))
        self._model = model
        def runner(
            frame: np.ndarray, prompts: list[str], params: dict[str, Any]
```

```
        ) -> list[RawDetection]:



            result = model.predict(
                frame,
                conf=params["conf"],
                iou=params["iou"],
                imgsz=params["imgsz"],
                max_det=params["max_det"],
                device=params["device"],
                verbose=False,
            )[0]
            out: list[RawDetection] = []
            if result.boxes is None:
                return out
            boxes = result.boxes.xyxy.detach().cpu().tolist()
            confs = result.boxes.conf.detach().cpu().tolist()
            class_ids = result.boxes.cls.detach().cpu().int().tolist()
            for bbox, conf, cid in zip(boxes, confs, class_ids, strict=True):
                out.append(
                    {
                        "prompt": prompts[cid],
                        "confidence": float(conf),
                        "bbox": [float(v) for v in bbox],
                    }
                )
            return out
        return runner
def _clamp_int(value: float, low: int, high: int) -> int:
    v = int(round(value))
    return low if v < low else (high if v > high else v)
def contract_classes() -> tuple[str, ...]:
    """本模块产出的类别集合（供对账测试直接引用，避免测试里再抄一遍）。"""
    return CONTRACT_CLASSES
# ===== edge/detector/test_world_detector.py =====
"""开放词汇检测器（WorldDetector）测试。
运行:
    cd edge/detector && python -m pytest test_world_detector.py -v
（也已加入 Makefile 的 test-edge 目标）
本文件不需要 torch / ultralytics
-----------------------------------
模型的加载路径靠注入 runner 绕开：契约、阈值、映射、ROI、降级
这些逻辑都不依赖真实权重，用假 runner 就能完整验证。
真实权重只在"跑一遍对照评测"时才需要（见 artifacts/metrics/）。
这里有一条测试专门守着一个真实的坑
-----------------------------------
test_world_output_is_dropped_by_cv_threshold_but_kept_by_world_threshold:
原方案打算让开放词汇通道沿用 cv 通道的 0.45 时序门限。实测数据表明
该通道历史最高置信度为 0.184（见 world_detector 模块头部的标定表），
套用 0.45 会让它一条检测都过不去，而现象只是"world 通道没结果"。
所以本文件把它钉成两条断言：0.45 下必空、world 门限下必通。
"""
```

```python
from __future__ import annotations


import ast
import sys
from pathlib import Path
from typing import Any
import numpy as np
import pytest
cv2 = pytest.importorskip("cv2", reason="需要 opencv-python-headless")
HERE = Path(__file__).resolve().parent
EDGE = HERE.parent
ROOT = EDGE.parent
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(EDGE / "simulator"))
import label_map  # noqa: E402
from detector import CLASS_NAMES, CvDetector  # noqa: E402
from label_map import (  # noqa: E402
    CALIBRATION_PROMPTS,
    CONTRACT_CLASSES,
    DEMO_DEFAULT_CLASSES,
    DEMO_LIVE_PROMPTS,
    MATCH_EXACT,
    MATCH_FALLBACK,
    PROMPT_TO_CLASS,
    normalize,
    unmapped_prompts,
)
from simulator import TemporalValidator  # noqa: E402
from world_detector import (  # noqa: E402
    BACKEND_NAME,
    DEFAULT_WORLD_CONFIG,
    PROJECT_ROOT,
    WorldDetector,
    resolve_weights_path,
)
# cv 通道的时序门限（0.45）。本文件里凡是出现它，都是为了证明"不能套用到 world 通道"。
CV_TEMPORAL_MIN_CONFIDENCE = 0.45
# ---------------------------------------------------------------------
# 合成帧与假 runner
# ---------------------------------------------------------------------
def _sea(seed: int = 0, size: tuple[int, int] = (360, 640)) -> np.ndarray:
    rng = np.random.default_rng(seed)
    base = np.zeros((size[0], size[1], 3), np.uint8)
    base[:] = (110, 100, 82)
    noise = rng.normal(0, 4, base.shape)
    return np.clip(base.astype(np.int16) + noise, 0, 255).astype(np.uint8)
def _fake_runner(raw_by_call: list[list[dict[str, Any]]]):
    """返回一个按调用次序吐结果的 runner（附带记录收到的参数）。
    语义：第 n 次调用返回 raw_by_call[n]；次数用完之后一律返回空。
    需要"连续多帧都有同一批检测"的用例（例如喂时序校验器），
    请把同一份 raw 重复若干次传进来，别只传一次。
```

```
    """



    calls: list[dict[str, Any]] = []
    def runner(frame, prompts, params):
        calls.append({"prompts": list(prompts), "params": dict(params), "shape": frame.shape[:2]})
        if len(calls) <= len(raw_by_call):
            return raw_by_call[len(calls) - 1]
        return []
    runner.calls = calls  # type: ignore[attr-defined]
    return runner
def _det(
    config: dict[str, Any] | None = None, raw: list[dict[str, Any]] | None = None
) -> WorldDetector:
    """构造一个注入了假 runner 的 WorldDetector。
    第一个参数是配置，第二个是假 runner 要吐出的原始检测
     （沿用上游 runner 的中间形态：{"prompt", "confidence", "bbox"}）。
    这样契约测试、阈值测试、映射测试可以各自只关心自己那一维。
    """
    return WorldDetector(config or {}, runner=_fake_runner([raw or []]))
def _det_box(
    conf: float = 0.30,
    bbox: tuple[float, float, float, float] = (100, 80, 200, 160),
    prompt: str = "plastic bottle",
    config: dict[str, Any] | None = None,
) -> WorldDetector:
    raw = [{"prompt": prompt, "confidence": conf, "bbox": list(bbox)}]
    return _det(config, raw=raw)
# ---------------------------------------------------------------------
# 1) 输出契约：与 CvDetector 完全一致
# ---------------------------------------------------------------------
class TestContract:
    def test_keys_are_exactly_the_contract_keys(self) -> None:
        det = _det_box()
        dets = det.detect(_sea())
        assert len(dets) == 1
        assert set(dets[0]) == {"class", "confidence", "bbox"}, (
            "输出契约的键必须与 CvDetector 完全一致，多一个少一个都会打破下游"
        )
    def test_types(self) -> None:
        d = _det_box().detect(_sea())[0]
        assert isinstance(d["class"], str)
        assert isinstance(d["confidence"], float)
        assert isinstance(d["bbox"], list) and len(d["bbox"]) == 4
        assert all(isinstance(v, int) for v in d["bbox"]), (
            "bbox 必须是整数像素坐标（与 CvDetector 一致，浮点会破坏下游网格归并）"
        )
    def test_class_is_always_in_contract(self) -> None:
        for prompt in (*DEMO_DEFAULT_CLASSES, *CALIBRATION_PROMPTS, *DEMO_LIVE_PROMPTS,
                       "完全不认识的词 xyzzy"):
            det = _det_box(prompt=prompt)
            for d in det.detect(_sea()):
```

```
            assert d["class"] in CLASS_NAMES, f"{prompt} → {d['class']} 越出四类契约"



def test_bbox_never_exceeds_frame(self) -> None:
    frame = _sea(size=(240, 320))
    for bbox in [(0, 0, 10, 10), (-50, -50, 100, 100), (300, 230, 500, 400)]:
        det = _det_box(bbox=bbox)
        for d in det.detect(frame):
            x1, y1, x2, y2 = d["bbox"]
            assert 0 <= x1 < x2 <= 320
            assert 0 <= y1 < y2 <= 240
def test_degenerate_box_is_dropped(self) -> None:
    for bbox in [(100, 100, 100, 200), (100, 100, 200, 100), (200, 100, 100, 200)]:
        det = _det_box(bbox=bbox)
        assert det.detect(_sea()) == [], f"退化框应当被丢弃：{bbox}"
        assert det.stats()["dropped_bad_box"] >= 1
def test_non_finite_values_are_dropped(self) -> None:
    raw = [{"prompt": "plastic bottle", "confidence": float("nan"),
            "bbox": [10, 10, 50, 50]}]
    det = _det(raw=raw)
    assert det.detect(_sea()) == []
    assert det.stats()["dropped_bad_box"] == 1
def test_output_feeds_cv_detector_contract_shape_identically(self) -> None:
    """两个检测器的输出必须能走同一段下游代码。
    做法：各自产出一次检测，断言键集合与 bbox 类型一致。
    "两个模块各自都通过测试"不代表它们接得上：这条就是接线的守卫。
    """
    world_det = _det_box(conf=0.9)
    world_out = world_det.detect(_sea())
    cv_det = CvDetector()
    frame = _sea()
    cv2.circle(frame, (320, 180), 18, (232, 235, 238), -1)
    for i in range(15):
        cv_det.detect(_sea(seed=i))
    cv_out = cv_det.detect(frame)
    assert world_out and cv_out, "两侧都应产出检测"
    assert set(world_out[0]) == set(cv_out[0])
    for out, name in ((world_out[0], "world"), (cv_out[0], "cv")):
        assert isinstance(out["bbox"], list) and len(out["bbox"]) == 4, name
        assert out["class"] in CLASS_NAMES, name
def test_deterministic_order(self) -> None:
    """同一输入两次跑结果必须逐字节一致（下游对账依赖这一点）。"""
    raw = [
        {"prompt": "plastic bottle", "confidence": 0.5, "bbox": [300, 200, 340, 240]},
        {"prompt": "foam debris", "confidence": 0.5, "bbox": [100, 80, 200, 160]},
        {"prompt": "fishing net", "confidence": 0.9, "bbox": [10, 10, 60, 90]},
    ]
    first = _det(raw=raw).detect(_sea())
    second = _det(raw=raw).detect(_sea())
    assert first == second
    assert [d["confidence"] for d in first] == [0.9, 0.5, 0.5]
def test_empty_frame_returns_empty(self) -> None:
```

```
        det = _det_box()



        assert det.detect(np.zeros((0, 0, 3), np.uint8)) == []
        assert det.stats()["calls"] == 0, "空帧不该计入调用数"
    def test_backend_name_is_declared(self) -> None:
        assert BACKEND_NAME == "world"
        assert WorldDetector.backend_name == "world"
# ---------------------------------------------------------------------
# 2) 标签映射（穷举）
# ---------------------------------------------------------------------
class TestLabelMap:
    def test_contract_classes_match_detector(self) -> None:
        """契约类别必须与 detector.CLASS_NAMES 逐项一致（第四处定义漂移守卫）。"""
        assert tuple(CONTRACT_CLASSES) == tuple(CLASS_NAMES)
    def test_every_mapping_value_is_a_contract_class(self) -> None:
        bad = {k: v for k, v in PROMPT_TO_CLASS.items() if v not in CONTRACT_CLASSES}
        assert not bad, f"映射表里有非契约类别：{bad}"
    def test_mapping_keys_are_normalized(self) -> None:
        """映射表的键必须已是归一化形式，否则永远匹配不到。"""
        for key in PROMPT_TO_CLASS:
            assert key == normalize(key), f"映射表键未归一化：{key!r}"
    def test_demo_default_classes_all_map_explicitly(self) -> None:
        for prompt in DEMO_DEFAULT_CLASSES:
            cls, kind = label_map.map_prompt(prompt)
            assert kind == MATCH_EXACT, f"{prompt} 没有明确归宿（回落 other 是缺陷，不是设计）"
            assert cls in CONTRACT_CLASSES
    def test_calibration_prompts_all_map_explicitly(self) -> None:
        for prompt in CALIBRATION_PROMPTS:
            _, kind = label_map.map_prompt(prompt)
            assert kind == MATCH_EXACT, f"{prompt} 没有明确归宿"
    def test_live_demo_prompts_all_map_explicitly(self) -> None:
        """现场加词用的词也必须都有归宿，否则演示时会"加了词却没反应"。"""
        for prompt in DEMO_LIVE_PROMPTS:
            _, kind = label_map.map_prompt(prompt)
            assert kind == MATCH_EXACT, f"{prompt} 没有明确归宿"
    def test_demo_default_classes_stay_in_sync_with_source_script(self) -> None:
        """漂移守卫：本模块抄了一份演示脚本的 DEFAULT_CLASSES。
        上游哪天加了一个提示词而 label_map 没跟上，这条测试直接失败——
        而不是等到跑了 200 帧发现某类恒为 0 再回头查。
        """
        script = ROOT / "scripts" / "detect_marine_demo.py"
        assert script.exists(), f"演示脚本不在预期位置：{script}"
        tree = ast.parse(script.read_text(encoding="utf-8"))
        found: tuple[str, ...] | None = None
        for node in tree.body:
            if not isinstance(node, ast.Assign):
                continue
            names = [t.id for t in node.targets if isinstance(t, ast.Name)]
            if "DEFAULT_CLASSES" in names:
                found = tuple(ast.literal_eval(node.value))
                break
```

```
        assert found is not None, f"{script} 里没找到 DEFAULT_CLASSES"



        assert found == DEMO_DEFAULT_CLASSES, (
            "演示脚本的提示词集已变化，label_map.DEMO_DEFAULT_CLASSES 需要同步：\n"
            f"  脚本：{found}\n  本模块：{DEMO_DEFAULT_CLASSES}"
        )
    def test_unknown_prompt_falls_back_but_is_observable(self) -> None:
        cls, kind = label_map.map_prompt("a completely unknown thing")
        assert (cls, kind) == ("other", MATCH_FALLBACK)
        assert not label_map.is_mapped("a completely unknown thing")
        assert unmapped_prompts(["plastic bottle", "???"]) == ["???"]
    def test_normalization_handles_case_and_spacing(self) -> None:
        assert normalize("  Plastic   Bottle ") == "plastic bottle"
        assert label_map.mapped_class("  PLASTIC   BOTTLE ") == "plastic"
    def test_all_seven_prompt_groups_have_a_home(self) -> None:
        """四类都必须有词可达：否则某一类永远检不出来。"""
        reached = set(PROMPT_TO_CLASS.values())
        assert reached == set(CONTRACT_CLASSES), f"这些类别没有任何提示词可达：{set(CONTRACT_CLASSE…
# ---------------------------------------------------------------------
# 3) 置信度标定：本通道绝不复用 cv 的 0.45
# ---------------------------------------------------------------------
class TestCalibration:
    def test_default_conf_is_not_the_temporal_gate(self) -> None:
        assert DEFAULT_WORLD_CONFIG["conf"] != CV_TEMPORAL_MIN_CONFIDENCE
        assert DEFAULT_WORLD_CONFIG["temporal_min_confidence"] != CV_TEMPORAL_MIN_CONFIDENCE
    def test_temporal_min_confidence_matches_configured_value(self) -> None:
        det = _det({"temporal_min_confidence": 0.12})
        assert det.temporal_min_confidence == pytest.approx(0.12)
    def test_temporal_min_confidence_falls_back_to_default(self) -> None:
        det = _det({})
        assert det.temporal_min_confidence == pytest.approx(
            DEFAULT_WORLD_CONFIG["temporal_min_confidence"]
        )
    def test_low_confidence_is_dropped_by_model_gate(self) -> None:
        gate = float(DEFAULT_WORLD_CONFIG["conf"])
        det = _det(
            raw=[
                {
                    "prompt": "plastic bottle",
                    "confidence": gate / 2,
                    "bbox": [10, 10, 60, 60],
                }
            ]
        )
        assert det.detect(_sea()) == []
        assert det.stats()["dropped_low_conf"] == 1
    def test_boundary_confidence_is_kept(self) -> None:
        gate = float(DEFAULT_WORLD_CONFIG["conf"])
        raw = [{"prompt": "plastic bottle", "confidence": gate, "bbox": [10, 10, 60, 60]}]
        det = _det(raw=raw)
        assert len(det.detect(_sea())) == 1, "恰好等于门限应当保留（与 CvDetector 的 >= 语义一致）"
```

```
    def test_world_output_is_dropped_by_cv_threshold_but_kept_by_world_threshold(self) -> None:
        """守住原方案的致命口径错误。
        用历史实测里的量级（0.16）造一条检测：
        - 套 cv 通道的 0.45 门限 → 时序校验一条都确认不了；
        - 用 world 通道自己的门限 → 能确认。
        """
        observed_conf = 0.16  # 见 world_detector 模块头部的标定表（历史最高 0.184）
        raw = [{"prompt": "foam debris", "confidence": observed_conf,
                "bbox": [300, 170, 340, 200]}]
        world_gate = float(DEFAULT_WORLD_CONFIG["temporal_min_confidence"])
        def confirm_count(min_confidence: float) -> int:
            # 同一批检测连续出现若干帧（时序校验要求 min_hits 次命中才确认）
            det = WorldDetector(
                {"temporal_min_confidence": min_confidence},
                runner=_fake_runner([raw] * 8),
            )
            validator = TemporalValidator(
                window_frames=15, min_hits=3, grid_size=64,
                min_confidence=min_confidence, match_distance=60.0,
                max_misses=5, min_iou=0.25, decimation=1,
            )
            total = 0
            for i in range(8):
                total += len(validator.push(det.detect(_sea(seed=i))))
            return total
        assert confirm_count(world_gate) > 0, (
            "world 通道在自己的门限下必须能确认事件，否则这条通道等于没接上"
        )
        assert confirm_count(CV_TEMPORAL_MIN_CONFIDENCE) == 0, (
            "若 0.45 下也能确认，说明标定表已失效，请重新标定后再改这条断言"
        )
# ---------------------------------------------------------------------
# 4) 优雅降级（缺依赖不能炸主线，也不能静默）
# ---------------------------------------------------------------------
class TestDegradation:
    def test_unavailable_when_deps_missing(self, monkeypatch: pytest.MonkeyPatch) -> None:
        import clip_shim
        monkeypatch.setattr(
            clip_shim, "clip_available", lambda: (False, "未安装 torch（测试构造）")
        )
        det = WorldDetector({"clip_dir": "不存在的目录"})
        assert det.available is False
        assert det.detect(_sea()) == []
        assert det.last_error is not None and "torch" in det.last_error
    def test_unavailable_is_counted_not_silent(self, monkeypatch: pytest.MonkeyPatch) -> None:
        import clip_shim
        monkeypatch.setattr(clip_shim, "clip_available", lambda: (False, "缺依赖"))
        det = WorldDetector({})
        det.detect(_sea())
        det.detect(_sea())
```

```
        stats = det.stats()

        assert stats["unavailable_calls"] == 2
        assert stats["kept"] == 0
        assert stats["available"] is False
    def test_missing_weights_reports_actionable_reason(self, monkeypatch: pytest.MonkeyPatch,
                                                       tmp_path: Path) -> None:
        det = WorldDetector({"clip_dir": str(tmp_path / "nope")})
        # 不 mock 任何东西：本机若真的没装 torch，答案就是"未安装 torch"；
        # 装了 torch 但缺权重，答案必须指向目录与权重文件。
        ok, reason = det._probe_environment()
        assert ok is False
        assert reason, "不可用时必须给出原因，不能只返回 False"
    def test_failed_availability_check_populates_last_error(
        self, monkeypatch: pytest.MonkeyPatch
    ) -> None:
        """只问 available 时，也必须能拿到原因。
        主程序在启动时先问 `available`，不可用就退出并打印 `last_error`。
        若 `available` 探到原因却没存下来，失败提示会变成"不可用：None"，
        等于没说：这是实现时真实踩到的坑。
        """
        import clip_shim
        monkeypatch.setattr(
            clip_shim, "clip_available", lambda: (False, "未安装 torch（构造）")
        )
        det = WorldDetector({})
        assert det.available is False
        assert det.last_error is not None and "torch" in det.last_error
    def test_load_succeeds_with_injected_runner(self) -> None:
        det = WorldDetector({}, runner=_fake_runner([[]]))
        assert det.load() is True
        assert det.last_error is None
    def test_load_reports_model_build_failure(self, monkeypatch: pytest.MonkeyPatch) -> None:
        det = WorldDetector({})
        # 依赖探测全绿（模拟"依赖齐、权重在"），但模型构建失败
        monkeypatch.setattr(det, "_probe_environment", lambda: (True, "构造：依赖齐备且权重就绪"))
        def boom() -> None:
            raise RuntimeError("模型加载失败：请把 torch 升到 2.6 以上")
        monkeypatch.setattr(det, "_build_ultralytics_runner", boom)
        assert det.load() is False
        assert det.last_error is not None and "torch" in det.last_error
    def test_available_can_be_true_while_load_fails(self, monkeypatch: pytest.MonkeyPatch) -> None:
        """依赖齐备 ≠ 模型能加载。这是真实踩到的坑（2026-09-27）。
        实测环境：`torch 2.5.1` + `ultralytics 8.4.35`。
        `clip_shim.environment_report()` 三项全绿（依赖就绪 / 权重就绪 / ultralytics 就绪），
        但 ultralytics 因 CVE-2025-32434 拒绝在 torch<2.6 上加载 `.pt`
        （原文：*require users to upgrade torch to at least v2.6*）。
        结果就是：`available == True` 而每帧返回 0 检出——
        只看 `available` 就会把"根本没跑起来"记成"没有检到东西"。
        所以主程序与对照脚本都必须两步都查：先 available，再 load()。
        """
```

```
        det = WorldDetector({})



        monkeypatch.setattr(det, "_probe_environment", lambda: (True, "构造：依赖齐备且权重就绪"))
        def boom() -> None:
            raise RuntimeError("torch 版本过低，无法加载 .pt")
        monkeypatch.setattr(det, "_build_ultralytics_runner", boom)
        assert det.available is True, "依赖探测这一层应当是绿的（这正是坑所在）"
        assert det.load() is False, "load() 必须把这类失败拦下来"
        assert det.last_error is not None
    def test_runner_exception_does_not_break_the_loop(self) -> None:
        """单帧推理抛异常不能让整个取流循环崩掉。"""
        def exploding_runner(frame, prompts, params):
            raise RuntimeError("模拟的一次模型崩溃")
        det = WorldDetector({}, runner=exploding_runner)
        assert det.detect(_sea()) == []
        assert det.last_error is not None and "本帧推理失败" in det.last_error
        assert det.stats()["unavailable_calls"] == 1
    def test_failed_probe_is_not_retried_every_frame(self, monkeypatch: pytest.MonkeyPatch) -> None:
        """失败后不重复尝试（否则每帧都要 import 一次并打日志）。"""
        import clip_shim
        calls = {"n": 0}
        def counting_probe():
            calls["n"] += 1
            return False, "缺依赖"
        monkeypatch.setattr(clip_shim, "clip_available", counting_probe)
        det = WorldDetector({})
        for _ in range(5):
            det.detect(_sea())
        assert calls["n"] == 1, f"环境探测应只执行一次，实际 {calls['n']} 次"
# ---------------------------------------------------------------------
# 5) 现场加词
# ---------------------------------------------------------------------
class TestSetPrompts:
    def test_set_prompts_changes_class_mapping(self) -> None:
        det = _det(raw=[{"prompt": "fishing net", "confidence": 0.3, "bbox": [10, 10, 90, 30]}])
        assert det.detect(_sea())[0]["class"] == "fishing_gear"
        det.set_prompts(["foam buoy"])
        assert "foam buoy" in det.prompts
    def test_set_prompts_records_new_prompts(self) -> None:
        det = _det({})
        det.set_prompts(["plastic bag", "seaweed"])
        assert det.prompts == ("plastic bag", "seaweed")
        assert det.stats()["prompts"] == ["plastic bag", "seaweed"]
    def test_set_prompts_rejects_empty(self) -> None:
        det = _det({})
        with pytest.raises(ValueError, match="提示词不能为空"):
            det.set_prompts([])
    def test_unknown_prompt_is_counted_as_fallback(self) -> None:
        det = _det(raw=[{"prompt": "utterly unknown object", "confidence": 0.3,
                         "bbox": [10, 10, 60, 60]}])
        dets = det.detect(_sea())
```

```
951         assert dets[0]["class"] == "other"



952         assert det.stats()["fallback_other"] == 1
953     def test_unmapped_prompts_surface_in_stats(self) -> None:
954         det = _det({})
955         det.set_prompts(["plastic bottle", "wonky prompt"])
956         assert det.stats()["unmapped_prompts"] == ["wonky prompt"]
957     def test_prompts_are_passed_to_runner(self) -> None:
958         runner = _fake_runner([[]])
959         det = WorldDetector({}, runner=runner)
960         det.detect(_sea())
961         assert runner.calls[0]["prompts"] == list(DEMO_DEFAULT_CLASSES)
962# ---------------------------------------------------------------------
963# 6) ROI
964# ---------------------------------------------------------------------
965class TestRoi:
966     def test_empty_roi_keeps_everything(self) -> None:
967         det = _det_box(bbox=(10, 10, 60, 60))
968         assert len(det.detect(_sea())) == 1
969     def test_box_center_outside_roi_is_dropped(self) -> None:
970         roi = [[0.0, 0.0], [0.5, 0.0], [0.5, 1.0], [0.0, 1.0]]  # 左半屏
971         det = _det_box(bbox=(500, 100, 560, 160))  # 中心在右半屏
972         det._roi = roi
973         assert det.detect(_sea()) == []
974         assert det.stats()["dropped_roi"] == 1
975     def test_box_center_inside_roi_is_kept(self) -> None:
976         roi = [[0.0, 0.0], [0.5, 0.0], [0.5, 1.0], [0.0, 1.0]]
977         det = _det_box(bbox=(20, 100, 80, 160))
978         det._roi = roi
979         assert len(det.detect(_sea())) == 1
980# ---------------------------------------------------------------------
981# 7) 配置对账与统计
982# ---------------------------------------------------------------------
983class TestConfigDrift:
984     def test_defaults_match_edge_config_yaml(self) -> None:
985         """代码默认值必须与 edge/config.yaml 的 detector.world 段逐项对账。
986         与 detector.py 的同类测试同一套路：参数写在 yaml 里、
987         实例化时漏传，会导致运维改了配置却不生效：且不报错。
988         """
989         import yaml
990         cfg = yaml.safe_load((EDGE / "config.yaml").read_text(encoding="utf-8"))
991         section = (cfg.get("detector") or {}).get("world")
992         assert section, "edge/config.yaml 里没有 detector.world 段"
993         mismatches: list[str] = []
994         for key, default in DEFAULT_WORLD_CONFIG.items():
995             if key not in section:
996                 mismatches.append(f"detector.world.{key} 在 config.yaml 中缺失")
997                 continue
998             actual = section[key]
999             if isinstance(default, bool) or isinstance(actual, bool):
1000                  if bool(default) != bool(actual):
```

```
                    mismatches.append(f"detector.world.{key}: 代码 {default} ≠ 配置 {actual}")


            elif isinstance(default, (int, float)) and isinstance(actual, (int, float)):
                if abs(float(default) - float(actual)) > 1e-6:
                    mismatches.append(f"detector.world.{key}: 代码 {default} ≠ 配置 {actual}")
            elif isinstance(default, list) and isinstance(actual, list):
                if [str(x) for x in default] != [str(x) for x in actual]:
                    mismatches.append(f"detector.world.{key}: 代码与配置的提示词集不一致")
        assert not mismatches, "默认值与配置不一致：\n  " + "\n  ".join(mismatches)
    def test_backend_default_is_cv(self) -> None:
        """现网行为不变：默认必须走 cv 通道。"""
        import yaml
        cfg = yaml.safe_load((EDGE / "config.yaml").read_text(encoding="utf-8"))
        assert (cfg.get("detector") or {}).get("backend") == "cv"
    def test_cv_defaults_section_untouched(self) -> None:
        """cv 通道的参数集不能被本通道的改动带偏。"""
        from detector import DEFAULT_CONFIG
        assert "backend" not in DEFAULT_CONFIG, (
            "detector.DEFAULT_CONFIG 是 cv 通道的参数集；"
            "开放词汇参数放在 world 段，不要混进来"
        )
        assert "world" not in DEFAULT_CONFIG
# ---------------------------------------------------------------------
# 权重路径解析（守一个真实踩到的坑）
# ---------------------------------------------------------------------
class TestWeightsPathResolution:
    """配置里的权重是裸文件名，而 ultralytics 按 cwd 找权重。
    后果（实测，不是推测）：从仓库根启动能找到本地文件；
    从 `edge/` 启动（`edge/main.py` 的常规用法）找不到 →
    ultralytics 静默转向联网下载，重试 3 次、每次都卡在
    `curl: (56) schannel: server closed abruptly`，启动挂住 2 分钟以上，
    且不抛出任何异常。现场演示正是从 edge/ 启动的。
    所以解析必须与 cwd 无关，缺权重必须显式失败而不是静默下载。
    """
    def test_bare_filename_resolves_against_repo_root_not_cwd(
        self, monkeypatch: pytest.MonkeyPatch, tmp_path: Path
    ) -> None:
        before = resolve_weights_path("yolov8s-worldv2.pt")
        monkeypatch.chdir(tmp_path)  # 换一个毫不相干的 cwd
        after = resolve_weights_path("yolov8s-worldv2.pt")
        assert before == after, "权重解析结果不得随当前工作目录变化"
        assert before == PROJECT_ROOT / "yolov8s-worldv2.pt"
        assert before.is_absolute()
    def test_absolute_path_is_preserved(self, tmp_path: Path) -> None:
        target = tmp_path / "some" / "weights.pt"
        assert resolve_weights_path(target) == target
    def test_shipped_weights_resolve_to_an_existing_file(self) -> None:
        """随包权重必须能在默认配置下被解析到一个真实存在的文件。
        这条同时守住两件事：权重没被挪走，以及默认 weights 没写成解析不到的值。
        """
        resolved = resolve_weights_path(DEFAULT_WORLD_CONFIG["weights"])
```

```
        assert resolved.exists(), (
            f"默认权重解析到 {resolved}，但该文件不存在："
            "从 edge/ 启动时 ultralytics 会转去联网下载并把启动卡住"
        )
    def _stub_deps_ok(self, monkeypatch: pytest.MonkeyPatch) -> None:
        """把依赖与 CLIP 目录探测打成"全绿"，让权重检查成为唯一的失败点。"""
        import clip_shim
        monkeypatch.setattr(clip_shim, "clip_available", lambda: (True, "依赖齐备（构造）"))
        monkeypatch.setattr(clip_shim, "clip_dir_status", lambda _p: (True, "CLIP 就绪（构造）"))
        monkeypatch.setattr(clip_shim, "ultralytics_available", lambda: (True, "ultralytics 就绪（…
    def test_missing_weights_and_download_disabled_fails_explicitly(
        self, monkeypatch: pytest.MonkeyPatch
    ) -> None:
        self._stub_deps_ok(monkeypatch)
        det = WorldDetector({"weights": "不存在的权重.pt", "allow_weights_download": False})
        ok, reason = det._probe_environment()
        assert ok is False
        # 原因必须给出解析后的绝对路径，否则运维不知道该把文件放哪
        assert str(resolve_weights_path("不存在的权重.pt")) in reason
        assert "allow_weights_download" in reason
    def test_missing_weights_with_download_enabled_is_not_blocked(
        self, monkeypatch: pytest.MonkeyPatch
    ) -> None:
        """显式允许下载时，可用性探测不应因为"本地没有"而拦下（改由 ultralytics 处理）。"""
        self._stub_deps_ok(monkeypatch)
        det = WorldDetector({"weights": "不存在的权重.pt", "allow_weights_download": True})
        ok, reason = det._probe_environment()
        assert ok is True, f"允许下载时不该在此拦下，实际原因：{reason}"
    def test_download_is_disabled_by_default(self) -> None:
        """默认必须禁止静默下载：现场网络不可预期，卡住比失败更糟。"""
        assert DEFAULT_WORLD_CONFIG["allow_weights_download"] is False
class TestCvChannelIsolation:
    """线上只读挂载约束（2026-09-27 核查发现）。
    `docker-compose.prod.yml` 把整个 `./edge` 只读挂载进线上容器
     （`- ./edge:/edge:ro`），后端有 3 处 `from edge.detector.detector import CvDetector`。
    这意味着：改动 edge/ 下的任何文件都会立刻被线上容器读到。
    所以开放词汇通道必须做到"只增不改"：
    - 不碰 `detector.py`（cv 通道实现）；
    - 不在 `__init__.py` 里引用本模块：否则后端那次 import 会连带把
      open-vocab 模块拉进平台进程，而它用的是扁平 import，在包上下文里会 ImportError，
      直接把线上后端拖挂。
    """
    def test_cv_detector_module_has_no_world_references(self) -> None:
        src = (HERE / "detector.py").read_text(encoding="utf-8")
        for token in ("world_detector", "WorldDetector", "DEFAULT_WORLD_CONFIG", "label_map"):
            assert token not in src, (
                f"detector.py 里出现了 {token}：cv 通道实现必须保持与开放词汇通道无关"
            )
    def test_package_init_does_not_pull_in_world_channel(self) -> None:
        init_src = (HERE / "__init__.py").read_text(encoding="utf-8")
```

```
        for token in ("world_detector", "label_map", "WorldDetector"):
            assert token not in init_src, (
                f"__init__.py 里出现了 {token}：后端的 `from edge.detector.detector import "
                "CvDetector` 会被它连带拉起，务必保持 __init__.py 不引用本模块"
            )
class TestStatsObservability:
    def test_stats_expose_every_drop_reason_separately(self) -> None:
        # 显式给配置，不吃随包默认值：
        #   本测试要构造"低于门限的框"，而"低多少算低"取决于 conf。
        #   原先这里用固定 0.02 去对比当时的默认 0.10，默认值一改
        #    (0.10 → 0.02) 这条 0.02 就不再低于门限，断言静默失效。
        #   把门限钉在测试里，测的才是"丢框计数"这件事本身。
        det = _det({"conf": 0.10, "temporal_min_confidence": 0.10}, raw=[
            {"prompt": "plastic bottle", "confidence": 0.02, "bbox": [10, 10, 60, 60]},
            {"prompt": "plastic bottle", "confidence": 0.30, "bbox": [10, 10, 10, 60]},
            {"prompt": "unknown thing", "confidence": 0.30, "bbox": [10, 10, 60, 60]},
            {"prompt": "foam debris", "confidence": 0.30, "bbox": [10, 10, 60, 60]},
        ])
        dets = det.detect(_sea(seed=1))
        stats = det.stats()
        assert len(dets) == 2
        assert stats["dropped_low_conf"] == 1
        assert stats["dropped_bad_box"] == 1
        assert stats["fallback_other"] == 1
        assert stats["raw"] == 4
        assert stats["kept"] == 2
    def test_default_conf_keeps_every_contract_class_reportable(self) -> None:
        """默认门限必须低于实测最紧类别的最高分。
        这条守的是一个真实发生过的缺陷：默认 conf 原为 0.10，
        而实测里 fishing_gear/plastic/foam 三类的最高分都低于 0.10——
        即在该门限下这个通道永远只会输出 other，
        现场"加一个词看它检出来"的演示必然毫无反应，
        而现象只是"加词没效果"，看不出是门限造成的。
        数据来源：artifacts/metrics/vision_backends_latest.json 的
        threshold_probe（5 张 4K 真实照片，门限压到 0.001）。
        它是这批图上的观测，不是真值；但足以约束默认门限：
        默认值一旦被调到 0.026 以上，本测试立刻失败并要求重新取证。
        """
        measured_class_top = {
            "other": 0.5824, "fishing_gear": 0.0514,
            "plastic": 0.0355, "foam": 0.0260,
        }
        conf = float(DEFAULT_WORLD_CONFIG["conf"])
        unreachable = sorted(c for c, top in measured_class_top.items() if top < conf)
        assert not unreachable, (
            f"默认 conf={conf} 会让这些契约类别永远不可报（实测最高分低于门限）："
            f"{unreachable}。这正是本次修掉的缺陷（原 0.10，四类里只有 other 可报）。"
        )
    def test_reset_clears_counters_but_keeps_prompts(self) -> None:
        det = _det_box()
```

```
        det.detect(_sea())



        det.set_prompts(["plastic bag"])
        det.reset()
        stats = det.stats()
        assert stats["calls"] == 0 and stats["kept"] == 0
        assert det.prompts == ("plastic bag",), "reset 不该动提示词"
    def test_stats_include_calibration_values(self) -> None:
        stats = _det({}).stats()
        assert stats["conf"] == pytest.approx(DEFAULT_WORLD_CONFIG["conf"])
        assert stats["temporal_min_confidence"] == pytest.approx(
            DEFAULT_WORLD_CONFIG["temporal_min_confidence"]
        )
# ===== edge/detector/clip_shim.py =====
"""为 YOLO-World 提供 CLIP 文本编码器垫片（从 scripts/detect_marine_demo.py 提取）。
背景
----
ultralytics 的 YOLO-World 在 model.set_classes(prompts) 时需要一个
OpenAI 的 clip 包来做文本编码。本仓库不想为此再装一个运行时，
于是用 transformers 版的 CLIP（openai/clip-vit-base-patch32）
临时顶替 clip 模块：只要 encode_text / tokenize / load
三个接口对得上，ultralytics 就察觉不到区别。
这个垫片原本只写在一份演示脚本里（scripts/detect_marine_demo.py）。
现在边缘侧也要用，所以提取成共享模块，避免出现"两份 shim 各自演化"。
依赖是可选的，导入必须安全
---------------------------
本模块顶层不导入 torch / transformers。在这台机器上，边缘测试环境
(.venv-analysis) 没有 torch，如果顶层导入，import world_detector
会直接炸掉 make test-edge。所以：
- clip_available()  只做探测，不导入
- install_clip_shim() 内部才导入，缺依赖时抛 ClipUnavailable 并带可执行的提示
这是本项目"可选依赖不能把主线拖下水"的一贯做法。
"""
from __future__ import annotations
import sys
import types
from pathlib import Path
from typing import Any
# CLIP 文本编码器的默认落盘位置（与 scripts/detect_marine_demo.py 的 --clip-dir 默认值一致）
DEFAULT_CLIP_DIR = Path.home() / ".cache" / "clip-vit-base-patch32"
# 需要存在的权重文件名。HF 上这个模型提供的是 pytorch_model.bin（不是 safetensors），
# 这也是为什么探测不能只看目录是否存在：目录存在但只有一个 .incomplete 分片，
# 是"下载到一半"的典型状态，必须当成不可用。
_CLIP_WEIGHT_FILES = ("pytorch_model.bin", "model.safetensors")
class ClipUnavailable(RuntimeError):
    """CLIP 文本编码器不可用（缺少依赖或权重）。"""
def clip_available() -> tuple[bool, str]:
    """探测 CLIP 垫片能否工作。返回 (可用, 原因)。
    只做 importlib 层面的探测与文件存在性检查，不加载模型，
    因此可以安全地在启动日志、配置校验里调用。
    """
```

```
    try:
        import torch  # noqa: F401, PLC0415
    except ImportError as exc:
        return False, f"未安装 torch ({exc}) "
    try:
        import transformers  # noqa: F401, PLC0415
    except ImportError as exc:
        return False, f"未安装 transformers ({exc}) "
    return True, "依赖就绪"
def clip_dir_status(clip_dir: Path | str | None = None) -> tuple[bool, str]:
    """检查 CLIP 权重目录是否完整（有真正的权重文件，不是半截下载）。
    为什么要单独查这一步：~/.cache/huggingface/hub 里出现
    blobs/*.incomplete 而快照目录只有 config.json，
    是"下载中断"的典型形态。此时目录存在，
    若不检查权重文件，报错会推迟到模型加载阶段，且信息指向完全不同的方向。
    """
    path = Path(clip_dir).expanduser() if clip_dir else DEFAULT_CLIP_DIR
    if not path.exists():
        return False, f"目录不存在：{path}"
    if not path.is_dir():
        return False, f"不是目录：{path}"
    for name in _CLIP_WEIGHT_FILES:
        if (path / name).exists():
            return True, f"权重就绪：{path / name}"
    return False, (
        f"目录存在但没有权重文件（{'/'.join(_CLIP_WEIGHT_FILES)}）：{path} "
        "—— 常见原因是下载未完成。国内网络建议走镜像，例如："
        "HF_ENDPOINT=https://hf-mirror.com python -c \"from huggingface_hub import "
        "snapshot_download; snapshot_download('openai/clip-vit-base-patch32', "
        f"local_dir=r'{path}')\""
    )
def install_clip_shim(clip_dir: Path | str | None = None) -> None:
    """把 transformers 版 CLIP 注册成 clip 模块（供 ultralytics 使用）。
    调用一次即可；重复调用是幂等的（会覆盖同名模块）。
    抛出 ClipUnavailable 表示依赖或权重缺失，调用方应据此走降级路径。
    """
    path = Path(clip_dir).expanduser() if clip_dir else DEFAULT_CLIP_DIR
    deps_ok, deps_reason = clip_available()
    if not deps_ok:
        raise ClipUnavailable(deps_reason)
    weights_ok, weights_reason = clip_dir_status(path)
    if not weights_ok:
        raise ClipUnavailable(weights_reason)
    import torch  # noqa: PLC0415
    from transformers import CLIPModel, CLIPTokenizer  # noqa: PLC0415
    class TransformersClip(torch.nn.Module):  # noqa: D101
        """Ultralytics YOLO-World 用到的最小 OpenAI CLIP 接口。"""
        def __init__(self, model: CLIPModel, device: torch.device) -> None:
            super().__init__()
            self.model = model.to(device)
```

```
            self.model.eval()



        def encode_text(
            self, token_ids: torch.Tensor, *_: Any, **__: Any
        ) -> torch.Tensor:
            token_ids = token_ids.to(self.model.device)
            return self.model.get_text_features(
                input_ids=token_ids,
                attention_mask=(token_ids != 0).long(),
            )
    tokenizer = CLIPTokenizer.from_pretrained(str(path))
    def tokenize(texts: str | list[str], truncate: bool = True) -> torch.Tensor:
        if isinstance(texts, str):
            texts = [texts]
        encoded = tokenizer(
            texts,
            padding="max_length",
            truncation=bool(truncate),
            max_length=77,
            return_tensors="pt",
        )
        return encoded["input_ids"]
    def load(
        _name: str,
        device: str | torch.device = "cpu",
        **_kwargs: Any,
    ) -> tuple[Any, None]:
        model = CLIPModel.from_pretrained(str(path))
        return TransformersClip(model, torch.device(device)), None
    clip_module = types.ModuleType("clip")
    clip_module.load = load  # type: ignore[attr-defined]
    clip_module.tokenize = tokenize  # type: ignore[attr-defined]
    sys.modules["clip"] = clip_module
def ultralytics_available() -> tuple[bool, str]:
    """探测 ultralytics 是否可用（同样不导入）。"""
    try:
        import ultralytics  # noqa: F401, PLC0415
        return True, f"ultralytics {ultralytics.__version__}"
    except ImportError as exc:
        return False, f"未安装 ultralytics ({exc}) "
def environment_report(clip_dir: Path | str | None = None) -> dict[str, Any]:
    """把"开放词汇通道能不能跑"的判定拆成可打印的一组事实。
    给启动自检和排障用：与其在跑完一帧之后看到空结果，
    不如一开始就把缺什么、缺在哪说清楚。
    """
    deps_ok, deps_reason = clip_available()
    weights_ok, weights_reason = clip_dir_status(clip_dir)
    ultra_ok, ultra_reason = ultralytics_available()
    return {
        "ready": deps_ok and weights_ok and ultra_ok,
        "deps": {"ok": deps_ok, "reason": deps_reason},
```

```
        "clip_weights": {
            "ok": weights_ok,
            "reason": weights_reason,
            "dir": str(Path(clip_dir).expanduser() if clip_dir else DEFAULT_CLIP_DIR),
        },
        "ultralytics": {"ok": ultra_ok, "reason": ultra_reason},
    }
# ===== edge/detector/label_map.py =====
"""开放词汇标签 → 四类契约类别的映射表。
为什么需要它
------------
YOLO-World 这类开放词汇检测器的输出类别是提示词原文
("plastic bottle" / "foam debris" / "discarded shoe" …)，
而本项目的输出契约只有四类（foam / plastic / fishing_gear / other，
见 edge/detector/detector.py 的 CLASS_NAMES）。
契约不允许"多出一类"：下游的时序校验（edge/simulator/simulator.py）、
MQTT 主题（docs/mqtt-topics.md）与平台侧四源对账
（scripts/check_contract_drift.py）都按四类写死。
所以开放词汇的自由文本必须在进入契约之前被映射回四类。
设计要点（都是本项目踩过坑之后定下的）
--------------------------------------
1. 显式表，不做模糊匹配。 映射来自 PROMPT_TO_CLASS 这张显式表，
   不做词干化、不做相似度猜测：那类"看起来像就映射"的隐式行为，
   出问题时无法在代码里指出是哪一行把 A 判成了 B。
2. 回落必须可观测。 未命中的词回落 other，但 map_prompt()
   同时返回 match_kind，调用方可以统计 fallback_other 的数量。
   静默把未知词变成 other，正是本项目反复出现的"静默失效"形状。
3. 穷举由测试守住。 DEMO_DEFAULT_CLASSES 与 CALIBRATION_PROMPTS
   是历史上真实用过的提示词全集；test_label_map.py 除了逐词断言，
   还用 AST 解析 scripts/detect_marine_demo.py 里的 DEFAULT_CLASSES，
   一旦上游加了新提示词而这里没跟上，测试直接失败。
本模块不导入 cv2 / torch / ultralytics，可单独测试。
"""
from __future__ import annotations
# ---------------------------------------------------------------------------
# 契约类别：必须与 edge/detector/detector.py 的 CLASS_NAMES 逐项一致。
# 这里刻意不 import detector（避免为了一个字符串列表拉进 cv2），
# 改用一条对账测试把两处钉在一起（test_label_map.py::test_contract_classes_match_detector）。
# ---------------------------------------------------------------------------
CONTRACT_CLASSES: tuple[str, ...] = ("foam", "plastic", "fishing_gear", "other")
# 未命中映射时的回落类别
UNMAPPED_CLASS = "other"
# match_kind 的取值
MATCH_EXACT = "exact"           # 提示词原文直接命中映射表
MATCH_FALLBACK = "fallback_other"  # 未命中，回落 other
# ---------------------------------------------------------------------------
# 提示词 → 四类。键必须是小写、空白已折叠的形式（见 normalize）。
#
# 分组依据是"现场怎么喊这个东西"，不是模型语义：
#   塑料类：有彩度、成型的塑料制品（瓶/袋/浮球/颗粒）
```

```
#   泡沫类：EPS 泡沫浮球及其碎片（白亮团状，当地治理优先级最高）



#   渔具类：废旧渔网、绳索、饵料包装
#   other：木板、藻团、生活杂物、鞋等兜底
# ---------------------------------------------------------------------------
PROMPT_TO_CLASS: dict[str, str] = {
    # ---- plastic ----
    "plastic bottle": "plastic",
    "plastic bottles": "plastic",
    "plastic waste": "plastic",
    "plastic debris": "plastic",
    "plastic bag": "plastic",
    "plastic bags": "plastic",
    "plastic cup": "plastic",
    "plastic pellet": "plastic",
    "nurdle": "plastic",
    "buoy": "plastic",
    "float": "plastic",
    # ---- foam ----
    "foam": "foam",
    "foam debris": "foam",
    "foam cup": "foam",
    "foam buoys": "foam",
    "styrofoam": "foam",
    "styrofoam cup": "foam",
    "broken styrofoam cup": "foam",
    "damaged foam cup": "foam",
    "polystyrene": "foam",
    "polystyrene cup": "foam",
    "eps": "foam",
    # ---- fishing_gear ----
    "fishing net": "fishing_gear",
    "fishing nets": "fishing_gear",
    "fishing gear": "fishing_gear",
    "net": "fishing_gear",
    "rope": "fishing_gear",
    "fishing rope": "fishing_gear",
    # ---- other ----
    "garbage": "other",
    "marine debris": "other",
    "discarded shoe": "other",
    "shoe": "other",
    "wood": "other",
    "wooden plank": "other",
    "seaweed": "other",
    "algae": "other",
}
# ---------------------------------------------------------------------------
# 历史上真实用过的提示词全集（穷举测试的靶子）
# ---------------------------------------------------------------------------
# 1) scripts/detect_marine_demo.py 的 DEFAULT_CLASSES（有 AST 对账测试守住同步）
```

```
DEMO_DEFAULT_CLASSES: tuple[str, ...] = (
    "plastic bottle",
    "plastic waste",
    "garbage",
    "fishing net",
    "foam debris",
    "marine debris",
    "discarded shoe",
)
# 2) artifacts/vision-preview/ 下四份 prompt 标定记录用过的提示词
CALIBRATION_PROMPTS: tuple[str, ...] = (
    # foam-prompt-calibration.json
    "foam",
    "styrofoam",
    "plastic pellet",
    "nurdle",
    # styrofoam-cup-calibration.json
    "styrofoam cup",
    "foam cup",
    "plastic cup",
    # styrofoam-cup-calibration-2.json
    "broken styrofoam cup",
    "damaged foam cup",
    "polystyrene cup",
    # fishing-net-calibration.json
    "net",
    "rope",
    "fishing gear",
)
# 3) 现场加词演示时会用到的词（评委随口说一个，系统应当能接住）
DEMO_LIVE_PROMPTS: tuple[str, ...] = (
    "buoy",
    "plastic bag",
    "wooden plank",
    "seaweed",
)
def normalize(prompt: str) -> str:
    """归一化提示词：去首尾空白、折叠内部连续空白、转小写。
    只做这三件事。不加复数还原、不加同义词扩展：那些需要词表外知识，
    会把"映射失败"变成"猜错了但没人知道"。
    """
    return " ".join(str(prompt).split()).lower()
def map_prompt(prompt: str) -> tuple[str, str]:
    """把一条开放词汇提示词映射到契约类别。
    返回 (class, match_kind)：
    - ("foam", "exact")：直接命中映射表
    - ("other", "fallback_other")：未命中，回落兜底类（可观测）
    调用方应把 fallback_other 计数暴露出来（WorldDetector.stats() 就是这么做的），
    否则"提示词写错了"会表现为"这一类检测不到"，排查方向完全被带偏。
    """
```

```
    key = normalize(prompt)



    cls = PROMPT_TO_CLASS.get(key)
    if cls is None:
        return UNMAPPED_CLASS, MATCH_FALLBACK
    return cls, MATCH_EXACT
def is_mapped(prompt: str) -> bool:
    """该提示词是否在映射表里有明确归宿（而非回落 other）。"""
    return normalize(prompt) in PROMPT_TO_CLASS
def mapped_class(prompt: str) -> str:
    """只要类别、不要 match_kind 时的便捷入口。"""
    return map_prompt(prompt)[0]
def unmapped_prompts(prompts: "list[str] | tuple[str, ...]") -> list[str]:
    """一次返回这组提示词里所有"会回落 other"的项。
    给配置校验用：装箱前先跑一遍，把没归宿的提示词打出来，
    而不是等到跑了 200 帧发现某类永远是 0 才回头查。
    """
    return [p for p in prompts if not is_mapped(p)]
# ===== edge/detector/test_metrics.py =====
"""指标口径测试：76% 抑制率只针对时序链路；precision/recall 零分母 = not_evaluated。
运行（仓库根）：
    python -m pytest edge/detector/test_metrics.py -q
冻结口径（/ 计划书 §6.1、§7.1）：
  1. 「76% 抑制率」= (fed - absorbed) / fed，分母是检测数，只针对时序链路
     （edge/simulator 的 TemporalValidator），不是检测器精度，禁止外推。
  2. precision/recall 任一分母为零 → 该指标为 None + not_evaluated，
     两个分母都为零 → 整项 not_evaluated，绝不生成虚假精度。
  3. 合成帧证据最高 E1-E2，禁止写成 E3/E4。
纯逻辑测试，不需要真实数据集；cv2 只用于端到端接线验证（缺失时跳过）。
"""
from __future__ import annotations
import sys
from pathlib import Path
import pytest
HERE = Path(__file__).resolve().parent
EDGE = HERE.parent
ROOT = EDGE.parent
sys.path.insert(0, str(EDGE / "simulator"))
sys.path.insert(0, str(ROOT / "ml" / "scripts"))
from simulator import TemporalValidator  # noqa: E402
from evaluate_opencv import (  # noqa: E402
    compute_class_metrics,
    resolve_evidence,
    suppression_rate,
)
# ---------------------------------------------------------------------------
# 76% 抑制率口径：只针对时序链路
# ---------------------------------------------------------------------------
class TestSuppressionRateTemporalScope:
    def test_formula_bounds(self) -> None:
        """抑制率恒在 0~1；fed<=0 时必须为 None（不得编造）。"""
```

```
from __future__ import annotations


import json
import os
import sys
import tempfile
import uuid
from pathlib import Path
import pytest
HERE = Path(__file__).resolve().parent
EDGE = HERE.parent
sys.path.insert(0, str(EDGE))
from device_sim.device import (  # noqa: E402
    DEVICE_SIM_SEQ_FILE_ENV,
    DeviceTwin,
    load_seq_from_file,
    save_seq_atomic,
)
from device_sim.protocol import FakeClock, SequentialIdFactory  # noqa: E402
from device_sim.transport import MemoryTransport  # noqa: E402
DEVICE = "RBT-SEQ-01"
@pytest.fixture
def seq_file() -> Path:
    """系统临时目录下的唯一文件路径（直接文件，无目录迭代）。
    绕过沙箱对 tmp_path 目录迭代的 PermissionError；文件级创建与
    清理在正常环境同样成立。返回的路径尚不存在（由测试决定是否创建）。
    """
    path = Path(tempfile.gettempdir()) / f"wp14e_seq_{os.getpid()}_{uuid.uuid4().hex[:10]}.json"
    try:
        yield path
    finally:
        try:
            path.unlink(missing_ok=True)
        except OSError:
            pass
def make_twin(seq_store_path, *, clock_start: float = 0.0) -> DeviceTwin:
    """构造带持久化路径的孪生（订阅 + 初始遥测）。"""
    twin = DeviceTwin(
        DEVICE,
        MemoryTransport(),
        clock=FakeClock(clock_start),
        id_factory=SequentialIdFactory("ack"),
        seq_store_path=seq_store_path,
    )
    twin.start()
    return twin
# ---------------------------------------------------------------------
# 默认不写盘（保持既有行为）
# ---------------------------------------------------------------------
class TestDefaultNoDiskWrite:
    def test_without_path_no_file_created(self, seq_file: Path, monkeypatch: pytest.MonkeyPatch) ->…
```

```
        """未配置路径（构造参数与环境变量都没有）→ 绝不落盘，行为与旧版一致。"""


        monkeypatch.delenv(DEVICE_SIM_SEQ_FILE_ENV, raising=False)
        twin = DeviceTwin(
            DEVICE,
            MemoryTransport(),
            clock=FakeClock(0.0),
            id_factory=SequentialIdFactory("ack"),
        )
        twin.start()
        assert twin.seq_store_path is None
        assert twin.seq == 1, "默认从 1 起计（首 tick）"
        twin.tick()
        twin.tick()
        assert twin.seq == 3
        assert not seq_file.exists(), "未配置路径时不得写盘"
# ---------------------------------------------------------------------
# 写盘 → 新进程重新加载 → 严格单调
# ---------------------------------------------------------------------
class TestCrossProcessMonotonic:
    def test_reload_continues_strictly_increasing(self, seq_file: Path) -> None:
        """写盘 → 新进程重新加载 → 新号 > 旧号；不回退、不重复已用序号。"""
        twin1 = make_twin(seq_file)
        twin1.tick()
        twin1.tick()
        twin1.tick()
        assert twin1.seq == 4            # start() 1 tick + 3 tick
        assert twin1.history["telemetry"][-1].seq == 4
        assert seq_file.exists()
        # 「进程重启」：全新 DeviceTwin 实例（新构造、重新加载文件）
        twin2 = make_twin(seq_file)          # start() 触发首 tick → 5
        assert twin2.seq == 5, "新进程必须从上次值续上并严格递增"
        assert twin2.history["telemetry"][0].seq == 5
        assert twin2.history["telemetry"][0].seq > twin1.seq
        # 已用过的 1..4 绝不重现
        assert all(t.seq > 4 for t in twin2.history["telemetry"])
        twin2.tick()
        assert twin2.seq == 6
    def test_multiple_restart_cycles_keep_increasing(self, seq_file: Path) -> None:
        """连续多次「重启」：每次新进程的首 tick 号都大于上一进程的最大号。"""
        prev_max = 0
        for i in range(3):
            twin = make_twin(seq_file)
            twin.tick()
            first_seq = twin.history["telemetry"][0].seq
            assert first_seq > prev_max, f"第 {i + 1} 次重启回退了序号"
            prev_max = first_seq
        assert prev_max >= 3
    def test_no_reuse_of_used_seq_after_reload(self, seq_file: Path) -> None:
        """重载后首个新序号必须 > 文件中的最大值，不得从 1 或旧值重来。"""
        twin1 = make_twin(seq_file)
```

```
        for _ in range(10):
            twin1.tick()
        persisted = json.loads(seq_file.read_text(encoding="utf-8"))["seq"]
        assert persisted == twin1.seq == 11
        # 「新进程」：先构造（加载文件）再启动，验证加载值与启动后自增
        twin2 = DeviceTwin(
            DEVICE,
            MemoryTransport(),
            clock=FakeClock(0.0),
            id_factory=SequentialIdFactory("ack"),
            seq_store_path=seq_file,
        )
        assert twin2.seq == 11, "构造时加载上次值，下一 tick 才自增"
        twin2.start()
        assert twin2.seq == 12
        assert twin2.history["telemetry"][0].seq == 12
# ---------------------------------------------------------------------
# 原子写盘
# ---------------------------------------------------------------------
class TestAtomicWrite:
    def test_no_tmp_leftover_and_valid_json(self, seq_file: Path) -> None:
        """原子写：落盘后无 `.tmp` 残留，内容为合法 JSON ({"seq": N}) 。"""
        twin = make_twin(seq_file)
        for _ in range(5):
            twin.tick()
        tmp_path = Path(f"{seq_file}.tmp")
        assert not tmp_path.exists(), "原子写不得留下临时文件"
        data = json.loads(seq_file.read_text(encoding="utf-8"))
        assert data == {"seq": twin.seq}
        assert twin.seq == 6
    def test_save_seq_atomic_roundtrip(self, seq_file: Path) -> None:
        """底层辅助函数：save → load 一致，且重复 save 幂等覆盖。"""
        assert save_seq_atomic(seq_file, 7) is True
        assert load_seq_from_file(seq_file) == (7, "ok")
        assert save_seq_atomic(seq_file, 8) is True
        assert load_seq_from_file(seq_file) == (8, "ok")
        assert not Path(f"{seq_file}.tmp").exists()
# ---------------------------------------------------------------------
# 降级：缺失 / 损坏 / 写盘失败
# ---------------------------------------------------------------------
class TestDegradation:
    def test_missing_file_is_normal_first_run(self, seq_file: Path) -> None:
        """文件缺失 = 首次运行：从 0 开始，下一 tick 为 1，不抛异常。"""
        assert not seq_file.exists()
        assert load_seq_from_file(seq_file) == (0, "missing")
        twin = make_twin(seq_file)
        assert twin.seq == 1
    def test_corrupt_file_resumes_from_last_parseable(self, seq_file: Path) -> None:
        """损坏文件：warning 降级，从最后一个可解析值继续。"""
        seq_file.write_text("garbage {{{ seq: 42 }}} trailing", encoding="utf-8")
```

```
        twin = DeviceTwin(
            DEVICE,
            MemoryTransport(),
            clock=FakeClock(0.0),
            id_factory=SequentialIdFactory("ack"),
            seq_store_path=seq_file,
        )
        assert twin.seq == 42, "损坏时取最后一个可解析值"
        twin.start()
        assert twin.seq == 43
    def test_totally_corrupt_file_starts_from_zero(self, seq_file: Path) -> None:
        """完全不可解析：warning 降级从 0 开始，首 tick 为 1，不抛异常。"""
        seq_file.write_text("!!! not a number !!!", encoding="utf-8")
        twin = DeviceTwin(
            DEVICE,
            MemoryTransport(),
            clock=FakeClock(0.0),
            id_factory=SequentialIdFactory("ack"),
            seq_store_path=seq_file,
        )
        assert twin.seq == 0
        twin.start()
        assert twin.seq == 1
        assert twin.history["telemetry"][0].seq == 1
    def test_save_failure_degrades_in_memory_only(self, seq_file: Path) -> None:
        """落盘失败（父目录不存在）：仅内存推进，遥测闭环不中断、不抛异常。"""
        target = Path(f"{seq_file}_nonexistent_dir") / "seq.json"
        twin = make_twin(target)
        twin.tick()
        assert twin.seq == 2            # start() 1 tick + 1 tick，内存仍推进
        assert twin.history["telemetry"][-1].seq == 2
        assert not target.exists()
# ---------------------------------------------------------------------
# 路径配置：环境变量与构造参数优先级
# ---------------------------------------------------------------------
class TestPathConfiguration:
    def test_env_var_configures_path(self, seq_file: Path, monkeypatch: pytest.MonkeyPatch) -> None:
        """环境变量 DEVICE_SIM_SEQ_FILE 可配置路径（不传构造参数）。"""
        monkeypatch.setenv(DEVICE_SIM_SEQ_FILE_ENV, str(seq_file))
        twin = DeviceTwin(
            DEVICE,
            MemoryTransport(),
            clock=FakeClock(0.0),
            id_factory=SequentialIdFactory("ack"),
        )
        twin.start()
        assert twin.seq_store_path == str(seq_file)
        assert seq_file.exists(), "环境变量配置的路径应生效并写盘"
    def test_constructor_path_takes_precedence_over_env(
        self, seq_file: Path, monkeypatch: pytest.MonkeyPatch
```

```
    ) -> None:



        """构造参数优先于环境变量。"""
        ctor_target = Path(f"{seq_file}_ctor.json")
        monkeypatch.setenv(DEVICE_SIM_SEQ_FILE_ENV, str(seq_file))
        twin = make_twin(ctor_target)
        assert twin.seq_store_path == str(ctor_target)
        assert ctor_target.exists()
        assert not seq_file.exists(), "构造参数应覆盖环境变量"
        ctor_target.unlink(missing_ok=True)
    def test_load_seq_from_file_float_json_ok(self, seq_file: Path) -> None:
        """JSON 中 seq 为整数形态浮点 (3.0) 也按 3 解析 (ok) 。"""
        seq_file.write_text('{"seq": 3.0}', encoding="utf-8")
        assert load_seq_from_file(seq_file) == (3, "ok")
# ===== edge/detector/test_detector.py =====
"""OpenCV 检测器测试：用合成帧验证，不需要真实摄像头与视频。
为什么能纯合成：检测器的输入是一帧 BGR 数组，输出是契约格式的检测列表。
只要能构造出"海面背景 + 已知目标"，就能断言它检出了什么。
这比"等出海采集再验证"快两个数量级，也是 OpenCV 路线相对 YOLO
最大的工程优势之一：不依赖数据也能证明它在工作。
运行:
    python -m pytest test_detector.py -v
"""
from __future__ import annotations
import ast
import re
import sys
from pathlib import Path
import numpy as np
import pytest
cv2 = pytest.importorskip("cv2", reason="需要 opencv-python-headless")
HERE = Path(__file__).resolve().parent
EDGE = HERE.parent
ROOT = EDGE.parent
sys.path.insert(0, str(HERE))
sys.path.insert(0, str(EDGE / "simulator"))
from detector import CLASS_NAMES, DEFAULT_CONFIG, CvDetector  # noqa: E402
from simulator import TemporalValidator  # noqa: E402
# 时序校验的门限 (时序过滤器的第一道闸，检测器给出的分数必须能跨过它)
MIN_CONFIDENCE = 0.45
# ---------------------------------------------------------------------
# 合成帧
# ---------------------------------------------------------------------
def _sea(seed: int = 0, size: tuple[int, int] = (360, 640)) -> np.ndarray:
    """灰蓝海面背景 (带纹理噪声) 。
    亮度刻意压在 V<150：真实海面在这个亮度以下，
    否则背景本身会被"白亮"颜色通道当成泡沫：那是最大的误报源。
    """
    rng = np.random.default_rng(seed)
    base = np.zeros((size[0], size[1], 3), np.uint8)
    base[:] = (110, 100, 82)  # BGR：偏灰蓝
```

```
    noise = rng.normal(0, 4, base.shape)
    return np.clip(base.astype(np.int16) + noise, 0, 255).astype(np.uint8)
def _with_foam(seed: int = 0, center: tuple[int, int] = (320, 180)) -> np.ndarray:
    frame = _sea(seed)
    cv2.circle(frame, center, 18, (232, 235, 238), -1)  # V≈238：白但没过曝
    return frame
def _with_plastic(seed: int = 0) -> np.ndarray:
    frame = _sea(seed)
    cv2.rectangle(frame, (420, 150), (480, 210), (220, 90, 60), -1)  # 蓝色块
    return frame
def _with_fishing_gear(seed: int = 0) -> np.ndarray:
    frame = _sea(seed)
    cv2.rectangle(frame, (150, 200), (240, 214), (55, 50, 45), -1)  # 暗色细长
    return frame
def _with_glare(seed: int = 0) -> np.ndarray:
    frame = _sea(seed)
    cv2.rectangle(frame, (500, 300), (530, 306), (255, 255, 255), -1)  # 极亮细条
    return frame
def _warm(detector: CvDetector, frames: int = 15) -> None:
    """让背景模型先学会"空海面"。"""
    for i in range(frames):
        detector.detect(_sea(seed=i))
def _first(detector: CvDetector, frame: np.ndarray) -> dict:
    dets = detector.detect(frame)
    assert dets, "期望至少检出一个目标，实际为空"
    return dets[0]
# ---------------------------------------------------------------------
# 基础检测行为
# ---------------------------------------------------------------------
class TestDetection:
    def test_calm_sea_detects_nothing(self) -> None:
        """平静海面（背景建模之后）不应有检测。"""
        det = CvDetector()
        _warm(det)
        assert det.detect(_sea(seed=99)) == [], "空海面被检出了目标：误报起点"
        print("  v 平静海面零检出")
    def test_foam_blob_is_detected(self) -> None:
        det = CvDetector()
        _warm(det)
        d = _first(det, _with_foam())
        assert d["class"] == "foam", f"白团被判成 {d['class']}"
        assert d["confidence"] >= MIN_CONFIDENCE, "置信度低于时序门限，会被直接丢弃"
        print(f"  v 泡沫：class={d['class']} conf={d['confidence']}")
    def test_colorful_plastic_is_detected(self) -> None:
        det = CvDetector()
        _warm(det)
        d = _first(det, _with_plastic())
        assert d["class"] == "plastic", f"彩色块被判成 {d['class']}"
        print(f"  v 塑料：class={d['class']} conf={d['confidence']}")
    def test_dark_elongated_is_fishing_gear(self) -> None:
```

```
    det = CvDetector()


    _warm(det)
    d = _first(det, _with_fishing_gear())
    assert d["class"] == "fishing_gear", f"暗色细长物被判成 {d['class']}"
    print(f"  v 渔具: class={d['class']} conf={d['confidence']}")
def test_bbox_is_valid_and_inside_frame(self) -> None:
    det = CvDetector()
    _warm(det)
    frame = _with_foam()
    h, w = frame.shape[:2]
    for d in det.detect(frame):
        x1, y1, x2, y2 = d["bbox"]
        assert len(d["bbox"]) == 4
        assert 0 <= x1 < x2 <= w, f"bbox 横向越界: {d['bbox']}"
        assert 0 <= y1 < y2 <= h, f"bbox 纵向越界: {d['bbox']}"
    print("  v bbox 全部落在画面内且有效")
def test_class_is_always_in_contract(self) -> None:
    """输出的类别必须落在四类别契约内: 否则平台侧枚举会拒收。"""
    det = CvDetector()
    _warm(det)
    for frame in (_with_foam(), _with_plastic(), _with_fishing_gear(), _with_glare()):
        for d in det.detect(frame):
            assert d["class"] in CLASS_NAMES, f"非法类别 {d['class']}"
    print("  v 类别全部在契约内")
def test_tiny_noise_is_ignored(self) -> None:
    """水花级别的小噪点不应成为候选。"""
    det = CvDetector()
    _warm(det)
    frame = _sea(seed=5)
    cv2.circle(frame, (300, 200), 3, (240, 240, 240), -1)  # 面积 ≈ 28 < min_area
    assert det.detect(frame) == [], "小水花被当成目标"
    print("  v 小噪点被过滤")
def test_static_target_still_detected(self) -> None:
    """停着不动的泡沫也必须持续检出。
    这是"颜色通道"存在的理由: 只用背景减除时, 一片停在镜头前的
    泡沫会在几十帧后被学成背景, 从此永不报警: 而泡沫是本项目
    最该抓的类别。这个测试锁住这个行为, 防止后人"优化"掉颜色通道。
    """
    det = CvDetector()
    _warm(det)
    for i in range(60):
        out = det.detect(_with_foam(seed=i))
        assert out and out[0]["class"] == "foam", f"第 {i} 帧丢失静止目标"
    print("  v 静止目标连续 60 帧仍在检出")
def test_glare_scores_below_foam(self) -> None:
    """海面反光必须显著低于真实泡沫的置信度。
    注意这是相对断言而不是"反光必须被压到 0.45 以下":
    高光与白泡沫在单帧特征上高度重合, 真把它们分开的是时序
    (高光闪现、凑不满 min_hits) 。这里只保证打分把它们排开。
    """
```

```
        det = CvDetector()


        _warm(det)
        foam = _first(det, _with_foam())["confidence"]
        det2 = CvDetector()
        _warm(det2)
        glare = _first(det2, _with_glare())["confidence"]
        assert foam - glare > 0.10, f"反光没被压下去：foam={foam} glare={glare}"
        print(f"  v 反光打分被压低：foam={foam} glare={glare}")
    def test_roi_excludes_outside_target(self) -> None:
        """ROI 之外的目标不参与检测（排除天空、岸边建筑、固定缆绳）。"""
        # 只在左半幅检测
        roi = [[0.0, 0.0], [0.6, 0.0], [0.6, 1.0], [0.0, 1.0]]
        det = CvDetector(roi=roi)
        _warm(det)
        assert det.detect(_with_foam(center=(550, 180))) == [], "ROI 外的目标混进来了"
        assert det.detect(_with_foam(center=(200, 180))), "ROI 内的目标丢了"
        print("  v ROI 生效")
    def test_confidence_within_declared_bounds(self) -> None:
        det = CvDetector()
        _warm(det)
        low = DEFAULT_CONFIG["confidence"]["min"]
        high = DEFAULT_CONFIG["confidence"]["max"]
        for frame in (_with_foam(), _with_plastic(), _with_fishing_gear()):
            for d in det.detect(frame):
                assert low <= d["confidence"] <= high, f"置信度越界：{d['confidence']}"
        print("  v 置信度落在 [min, max] 内")
# ---------------------------------------------------------------------
# 契约一致性（第四处类别定义不能漂移）
# ---------------------------------------------------------------------
class TestContractConsistency:
    def test_class_names_match_ai_server(self) -> None:
        """与 backend/app/services/ai/server.py 的 CLASS_NAMES 逐项一致。"""
        path = ROOT / "backend" / "app" / "services" / "ai" / "server.py"
        tree = ast.parse(path.read_text(encoding="utf-8"))
        found: list[str] | None = None
        for node in ast.walk(tree):
            if isinstance(node, ast.Assign):
                for t in node.targets:
                    if isinstance(t, ast.Name) and t.id == "CLASS_NAMES":
                        found = [e.value for e in node.value.elts]  # type: ignore[union-attr]
        assert found is not None, "没解析到 CLASS_NAMES：锚点失效，守卫已空转"
        assert found == CLASS_NAMES, f"AI 服务 {found} ≠ 检测器 {CLASS_NAMES}"
        print(f"  v 与 AI 服务一致：{CLASS_NAMES}")
    def test_class_names_match_dataset_yaml(self) -> None:
        """与 ml/configs/seasight.yaml 的 names（即模型类别索引）一致。"""
        import yaml  # noqa: PLC0415
        cfg = yaml.safe_load((ROOT / "ml" / "configs" / "seasight.yaml").read_text(encoding="utf-8"…
        names = cfg["names"]
        assert len(names) == len(CLASS_NAMES), f"yaml 里 {len(names)} 类，代码里 {len(CLASS_NAMES)}…
        for idx, name in sorted(names.items()):
```

```
            assert CLASS_NAMES[idx] == name, f"索引{idx}应为{name}，实际{CLASS_NAMES[idx]}"



        print("  v 与数据集 yaml 一致（顺序即模型类别索引）")
    def test_defaults_match_edge_config_yaml(self) -> None:
        """代码默认值必须与 edge/config.yaml 逐项对账。
        项目踩过的坑：参数写在 yaml 里，但实例化时漏传，导致运维改了
        配置却不生效：且不报错。这里把每个默认值都和 yaml 对一遍。
        """
        import yaml  # noqa: PLC0415
        cfg = yaml.safe_load((EDGE / "config.yaml").read_text(encoding="utf-8"))
        section = cfg.get("detector")
        assert section, "config.yaml 里没有 detector 段"
        mismatches: list[str] = []
        def walk(default: object, actual: object, path: str) -> None:
            if isinstance(default, dict):
                for key, value in default.items():
                    if not isinstance(actual, dict) or key not in actual:
                        mismatches.append(f"{path}.{key} 在 config.yaml 中缺失")
                        continue
                    walk(value, actual[key], f"{path}.{key}")
            elif isinstance(actual, bool) or isinstance(default, bool):
                if bool(default) != bool(actual):
                    mismatches.append(f"{path}: 代码{default} ≠ 配置{actual}")
            elif isinstance(default, (int, float)) and isinstance(actual, (int, float)):
                if abs(float(default) - float(actual)) > 1e-6:
                    mismatches.append(f"{path}: 代码{default} ≠ 配置{actual}")
        walk(DEFAULT_CONFIG, section, "detector")
        assert not mismatches, "默认值与配置不一致：\n  " + "\n  ".join(mismatches)
        print(f"  v {len(DEFAULT_CONFIG)} 个配置段与 config.yaml 对账通过")
# ---------------------------------------------------------------------
# 接线：检测器 → 时序校验器
# ---------------------------------------------------------------------
class TestWiring:
    def test_detector_output_feeds_temporal_validator(self) -> None:
        """检测器的输出必须能直接喂给时序校验器并被确认。
        这是"实现了 N 步、只接上了 N-2 步"那类缺陷的守卫：
        两个模块各自都通过测试，不代表它们接得上。
        """
        det = CvDetector()
        _warm(det)
        validator = TemporalValidator(decimation=1)  # 关抽帧，只看接线是否通
        confirmed: list[dict] = []
        for i in range(6):
            frame = _sea(seed=100 + i)
            cv2.circle(frame, (320 + i * 4, 180 + i * 3), 18, (232, 235, 238), -1)
            confirmed.extend(validator.push(det.detect(frame)))
        assert confirmed, "检测器与时序校验器没接上：连续 6 帧未确认任何事件"
        assert confirmed[0]["class"] == "foam"
        assert confirmed[0]["confidence"] >= MIN_CONFIDENCE
        print(f"  v 端到端接通：{len(confirmed)} 条确认事件（class={confirmed[0]['class']}）")
    def test_single_glare_frame_never_confirms(self) -> None:
```

```
        """反光只在单帧闪现 → 不该被确认为事件（时序压制生效）。"""



        det = CvDetector()
        _warm(det)
        validator = TemporalValidator(decimation=1)
        validator.push(det.detect(_with_glare()))
        for i in range(10):
            validator.push(det.detect(_sea(seed=200 + i)))
        # stats 是 property，不是方法
        assert validator.stats["confirmed"] == 0, "单帧反光被确认成事件了"
        print("  v 单帧反光未被确认")
if __name__ == "__main__":
    raise SystemExit(pytest.main([__file__, "-v", "-s"]))
# ===== edge/simulator/simulator.py =====
#!/usr/bin/env python3
"""探海灵眸 Oceanus — 边缘感知模拟器。
模拟边缘盒（岸基摄像头旁的计算单元）的完整行为，用于在没有真实
设备的情况下驱动平台演示。它复刻了真实边缘盒里几个容易做错的关键点：
1. 时序校验降误报
   浪花是瞬态的（出现一两帧就消失），垃圾是持续的。
   滑动窗口内连续 N 帧在同一网格命中，才升级为「确认事件」。
   不做这一步，海面视频每分钟会产生上百条误报。
2. seq 单调递增 + 本地持久化
   平台用 (device_id, seq) 唯一约束做幂等去重。
   seq 必须严格单调，且崩溃重启后从上次值续上，否则要么漏判重、要么全被拒。
3. 断网补传
   渔港 4G 信号不稳。断网期间事件进本地环形队列，恢复后按原 seq 顺序补发。
   不模拟这条链路，平台的判重与补派逻辑就永远测不到。
4. QoS1 重复投递
   --dup-rate 参数故意重复投递，验证平台幂等是否真的生效。
用法见同目录 README.md。
"""
from __future__ import annotations
import argparse
import json
import os
import random
import signal
import sys
import time
import uuid
from collections import defaultdict, deque
from dataclasses import dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Iterable
# ---------------------------------------------------------------------------
# 依赖与常量
# ---------------------------------------------------------------------------
#
# 依赖导入放在此处但不在此处退出：
# TemporalValidator 是纯逻辑类，时序校验的单元测试不该被迫装上
```

```
# pyyaml / paho-mqtt。真正缺失依赖时，在 main() 里给出明确报错。



# ---------------------------------------------------------------------
try:
    import yaml
except ImportError:   # pragma: no cover
    yaml = None       # type: ignore[assignment]
try:
    import paho.mqtt.client as mqtt
except ImportError:   # pragma: no cover
    mqtt = None       # type: ignore[assignment]
def _require_deps() -> None:
    """运行模拟器前检查依赖，缺失时给出明确的安装命令。"""
    missing: list[str] = []
    if yaml is None:
        missing.append("pyyaml")
    if mqtt is None:
        missing.append("paho-mqtt")
    if missing:
        print(
            f"[错误] 缺少依赖：{', '.join(missing)}\n"
            f"       请执行：pip install {' '.join(missing)}",
            file=sys.stderr,
        )
        sys.exit(1)
WASTE_CLASSES = ("foam", "plastic", "fishing_gear", "other")
CLASS_LABELS = {
    "foam": "泡沫类",
    "plastic": "塑胶类",
    "fishing_gear": "渔具类",
    "other": "其他",
}
# 各垃圾类别的视觉特征（用于生成合理的检测框与置信度）
CLASS_VISUAL: dict[str, dict[str, Any]] = {
    # (框尺寸范围 px，置信度基准)
    "foam": {"size": (24, 70), "conf": (0.72, 0.95)},
    "plastic": {"size": (30, 90), "conf": (0.65, 0.90)},
    "fishing_gear": {"size": (60, 200), "conf": (0.68, 0.92)},
    "other": {"size": (20, 120), "conf": (0.48, 0.75)},
}
STREAM_STATE_FILE = ".simulator_state.json"
FRAME_W, FRAME_H = 1920, 1080
DEFAULT_CONFIG = Path(__file__).with_name("config.yaml")
# ---------------------------------------------------------------------
# 时序校验器：降误报的第一道闸
# ---------------------------------------------------------------------
class _Track:
    """被跟踪的一个候选目标（可能真、可能是噪声）。"""
    __slots__ = (
        "track_id",
        "cls",
```

```
        "bbox",
        "confidence",
        "hits",
        "misses",
        "last_cell",
        "last_hit_frame",
        "confirmed",
    )
    def __init__(self, track_id: int, det: dict[str, Any], cell: tuple[int, int]) -> None:
        self.track_id = track_id
        self.cls = det["class"]
        self.bbox = list(det["bbox"])
        self.confidence = det["confidence"]
        self.hits = 1
        self.misses = 0
        self.last_cell = cell
        self.last_hit_frame = 0
        self.confirmed = False
    def update(self, det: dict[str, Any], cell: tuple[int, int]) -> None:
        self.bbox = list(det["bbox"])
        self.cls = det["class"]
        self.confidence = det["confidence"]
        self.hits += 1
        self.misses = 0
        self.last_cell = cell
class TemporalValidator:
    """时序校验器：降误报的第一道闸。
    核心判断：浪花是瞬态的，垃圾是持续的。
    ── 为什么不能按「像素格网」统计命中 ──
    每帧的检测框像素位置都在抖动，即使同一块泡沫，也不可能两帧落在
    完全相同的 64px 格网里。若按格网统计，"连续 N 帧命中"永远凑不满，
    校验就形同虚设。
    ── 正确模型 ──
    把目标而不是像素格作为跟踪单元：
    相邻帧中中心点距离足够近、类别相同的检测，视为同一目标，
    维护它自己的命中计数；连续命中达到 min_hits 才确认为真事件。
    这对应真实边缘盒里「检测 → 匈牙利匹配/光流跟踪」那一步。
    同时：命中不足的目标按帧衰减淘汰，单帧噪声自然被饿死。
    """
    def __init__(
        self,
        window_frames: int = 15,
        min_hits: int = 3,
        grid_size: int = 64,
        min_confidence: float = 0.45,
        match_distance: float = 60.0,
        max_misses: int = 5,
        min_iou: float = 0.25,
        decimation: int = 4,
    ) -> None:
```

```
        # 这里的默认值必须与 docs/software-design.md §2.3 参数表逐项一致。
        #   曾经 max_misses 在文档里写 5、代码里写 6、测试里硬写 6——
        #   三方不一致却从未被发现，因为测试跟着实现走，永远"通过"。
        #   真实边缘盒上这个值决定「丢失几帧后放弃目标」：
        #   偏小 → 目标被浪遮挡就丢，同一片垃圾重复上报；
        #   偏大 → 噪声目标赖着不走，占跟踪槽位。
        #   改这里请同步改文档与 config.yaml（config 未配时以此默认为准）。
        self.window_frames = window_frames
        self.min_hits = min_hits
        self.grid_size = grid_size
        self.min_confidence = min_confidence
        # 同一目标在相邻帧间的最大中心位移（像素），1080p 下约画面宽度 3%
        self.match_distance = match_distance
        # 连续多少帧没匹配上就淘汰（0.6 秒，容忍短暂遮挡）
        self.max_misses = max_misses
        # 关联所需的最小框重叠度：单靠距离门限挡不住随机噪声
        self.min_iou = min_iou
        self._tracks: list[_Track] = []
        self._next_track_id = 1
        self._total_frames = 0
        # 统计口径：三个计数必须同单位，否则算出来的抑制率会 >100%。
        #   _fed：本窗口喂进校验器的检测总数（分母）
        #   _lowered：因置信度不足被丢弃的检测数
        #   _absorbed：被某个 track 关联吸收的检测数（即"有效命中"）
        #   抑制率 = (fed - absorbed) / fed
        #          = 从未被任何 track 稳定跟踪的检测占比
        #   这比"累计淘汰 track 数"更严谨：一个 track 淘汰只算一次，
        #   而它可能只对应 1 个检测，也可能对应几十个：无法与检测数直接相除。
        self._fed = 0
        self._lowered = 0
        self._absorbed = 0
        # 抽帧推理模拟：真实边缘盒不是逐帧跑模型，而是隔几帧推一帧
        #   （算力就这么点，8fps 对水面漂浮物足够）。
        #   抽帧的意义不止省算力：它天然压噪声：只在单帧闪现的浪花
        #   被抽中的概率仅 1/N，根本凑不满 min_hits 就被饿死；
        #   而持续存在的垃圾几乎每帧都在，抽中照样连续命中。
        #   所以抑制率会比逐帧模式高得多，也更贴近真实产出。
        self.decimation = max(1, int(decimation))
        self._decimation_tick = 0
        self._sweep_tick = 0
    def _cell(self, bbox: list[int]) -> tuple[int, int]:
        """把检测框中心映射到格网坐标（用于上报冷却，不用于跟踪）。"""
        cx = (bbox[0] + bbox[2]) / 2
        cy = (bbox[1] + bbox[3]) / 2
        return int(cx // self.grid_size), int(cy // self.grid_size)
    @staticmethod
    def _center(bbox: list[int]) -> tuple[float, float]:
        return (bbox[0] + bbox[2]) / 2, (bbox[1] + bbox[3]) / 2
    @staticmethod
    def _iou(a: list[int], b: list[int]) -> float:
```

```
        """两个检测框的交并比。"""
        ax1, ay1, ax2, ay2 = a
        bx1, by1, bx2, by2 = b
        ix1, iy1 = max(ax1, bx1), max(ay1, by1)
        ix2, iy2 = min(ax2, bx2), min(ay2, by2)
        iw, ih = max(0, ix2 - ix1), max(0, iy2 - iy1)
        inter = iw * ih
        if inter == 0:
            return 0.0
        area_a = (ax2 - ax1) * (ay2 - ay1)
        area_b = (bx2 - bx1) * (by2 - by1)
        union = area_a + area_b - inter
        return inter / union if union > 0 else 0.0
    def _match(self, det: dict[str, Any]) -> _Track | None:
        """为当前检测找最近的同类未淘汰目标。
        关联条件同时满足才认为是同一目标：
        1. 类别相同
        2. 中心位移 < match_distance
        3. IoU > min_iou
        为什么必须加 IoU：只有中心距离门限的话，画面里随机落点的
        瞬态噪声偶尔会"碰巧"靠近某个跟踪目标，把它的命中计数续上去，
        于是噪声也能凑满 min_hits 被误确认。加上重叠度要求后，
        随机噪声几乎不可能连续命中同一个目标：这正是两级门限的价值。
        """
        cx, cy = self._center(det["bbox"])
        best: _Track | None = None
        best_dist = self.match_distance
        for track in self._tracks:
            if track.cls != det["class"]:
                continue
            tx, ty = self._center(track.bbox)
            dist = ((cx - tx) ** 2 + (cy - ty) ** 2) ** 0.5
            if dist >= best_dist:
                continue
            if self._iou(track.bbox, det["bbox"]) < self.min_iou:
                continue
            best_dist = dist
            best = track
        return best
    def push(self, detections: list[dict[str, Any]]) -> list[dict[str, Any]]:
        """喂入一帧的原始检测结果，返回通过时序校验的确认检测。
        每帧都必须调用（即使 detections 为空），否则目标无法衰减淘汰。
        """
        self._total_frames += 1
        self._fed += len(detections)
        # 置信度过滤
        candidates: list[dict[str, Any]] = []
        for det in detections:
            if det["confidence"] < self.min_confidence:
                self._lowered += 1
```

```
                continue


            candidates.append(det)
        # 模拟抽帧推理：只对一部分帧真正跑检测，其余帧返回空。
        #   这样噪声目标（只在单帧闪现）几乎不可能被连续命中，
        #   而稳定目标（持续存在）在抽中的帧里依然会被反复匹配到。
        self._decimation_tick += 1
        if self._decimation_tick % self.decimation != 0:
            return []
        # 已匹配的目标集合（避免一个目标被两个检测重复匹配）
        matched: set[int] = set()
        for det in candidates:
            track = self._match(det)
            if track is not None and track.track_id not in matched:
                track.update(det, self._cell(det["bbox"]))
                track.last_hit_frame = self._total_frames
                matched.add(track.track_id)
                self._absorbed += 1        # 被稳定目标吸收：不是误报
            else:
                # 新目标入池
                self._tracks.append(
                    _Track(self._next_track_id, det, self._cell(det["bbox"]))
                )
                self._tracks[-1].last_hit_frame = self._total_frames
                self._next_track_id += 1
        # 判定确认：命中数达标且尚未上报过
        confirmed: list[dict[str, Any]] = []
        for track in self._tracks:
            if track.hits >= self.min_hits and not track.confirmed:
                track.confirmed = True
                confirmed.append(
                    {
                        "class": track.cls,
                        "confidence": track.confidence,
                        "bbox": list(track.bbox),
                        "_track_id": track.track_id,
                        "_hits": track.hits,
                    }
                )
        return confirmed
    def sweep(self, decimation: int = 4) -> int:
        """周期性结算未被命中的 track，返回本次淘汰的目标数。
        为什么需要它——
        真实推理是抽帧跑的（每 decimation 帧推一帧），而 miss 衰减
        必须按真实时间推进。若只在 push() 里衰减，miss 计的是"推理帧数"
        而不是"时间"，抽帧一开，噪声目标就要等 4 倍时间才被淘汰。
        做法：每帧调用本函数，累计到 decimation 次才真正走一次衰减，
        于是 miss 的口径重新对齐到"被跳过的帧"，语义与真实流水线一致。
        """
        self._sweep_tick = getattr(self, "_sweep_tick", 0) + 1
        if self._sweep_tick < decimation:
```

```
            return 0



        self._sweep_tick = 0
        before = len(self._tracks)
        # 本轮被命中的目标清零 miss：用 last_hit_frame 判断"本轮是否被更新过"
        survivors: list[_Track] = []
        for track in self._tracks:
            if track.last_hit_frame == self._total_frames:
                track.misses = 0
                survivors.append(track)
                continue
            track.misses += 1
            if track.misses > self.max_misses:
                continue
            survivors.append(track)
        self._tracks = survivors
        return before - len(survivors)
    def reset(self) -> None:
        """清空所有跟踪状态。"""
        self._tracks.clear()
        self._next_track_id = 1
        self._sweep_tick = 0
    @property
    def stats(self) -> dict[str, int]:
        """当前统计快照。
        `suppressed` 是从未被稳定跟踪的检测数，即被时序校验挡下的误报。
        口径与 `fed` 一致（都是"检测数"），所以抑制率恒在 0~100%。
        """
        return {
            "frames": self._total_frames,
            "tracked": len(self._tracks),
            "confirmed": sum(1 for t in self._tracks if t.confirmed),
            "fed": self._fed,
            "absorbed": self._absorbed,
            "low_confidence": self._lowered,
            "suppressed": max(0, self._fed - self._absorbed),
        }
    def drain_stats(self) -> dict[str, int]:
        """读取并清零本窗口的计数（供全局统计累加，避免重复计数）。"""
        out = self.stats
        self._fed = 0
        self._absorbed = 0
        self._lowered = 0
        return out
# ---------------------------------------------------------------------
# 断网补传缓存
# ---------------------------------------------------------------------
class OutboxBuffer:
    """本地补传队列（断网时缓存事件，恢复后按序补发）。
    真实边缘盒上这是 SQLite 表；模拟器用内存 deque 表达同一语义。
    上限策略：满了丢最旧：宁可丢历史，不能阻塞新事件。
```

```
    """



    def __init__(self, max_size: int = 1000) -> None:
        self.max_size = max_size
        self._queue: deque[tuple[str, str, int]] = deque(maxlen=max_size)
        self.dropped = 0
    def put(self, topic: str, payload: dict[str, Any], qos: int) -> None:
        if len(self._queue) == self.max_size:
            self.dropped += 1
        self._queue.append((topic, json.dumps(payload, ensure_ascii=False), qos))
    def flush(self, publish_fn, batch: int = 20) -> int:
        """把缓存的事件补传出去，返回补传条数。"""
        sent = 0
        while self._queue and sent < batch:
            topic, payload, qos = self._queue.popleft()
            publish_fn(topic, payload, qos)
            sent += 1
        return sent
    def __len__(self) -> int:
        return len(self._queue)
# ---------------------------------------------------------------------
# 设备模拟
# ---------------------------------------------------------------------
@dataclass
class SimDevice:
    """一台被模拟的设备。"""
    device_id: str
    name: str
    device_type: str
    lng: float
    lat: float
    profile: dict[str, float]
    rate: float = 1.0
    patrol_radius_deg: float = 0.0
    # 运行时状态
    seq: int = 0
    battery: int = field(default_factory=lambda: random.randint(75, 98))
    online: bool = True
    validator: TemporalValidator | None = None
    # 同一格网的上报冷却：{(cell): last_report_ts}
    _last_report: dict[tuple[int, int], float] = field(default_factory=dict)
    def random_location(self, rng: random.Random) -> tuple[float, float]:
        """事件落点（带小幅漂移，模拟垃圾随潮流移动）。"""
        if self.patrol_radius_deg > 0:
            # 无人机/广域设备：巡飞范围内随机
            return (
                self.lng + rng.uniform(-self.patrol_radius_deg, self.patrol_radius_deg),
                self.lat + rng.uniform(-self.patrol_radius_deg, self.patrol_radius_deg),
            )
        # 固定摄像头：画面覆盖范围内的近岸水域（约 ±0.004 度 ≈ 400m）
        return (
```

```
            self.lng + rng.uniform(-0.004, 0.004),
            self.lat + rng.uniform(-0.004, 0.004),
        )
    def pick_class(self, rng: random.Random) -> str:
        """按该点位的垃圾构成抽取类别。"""
        classes = list(self.profile.keys())
        weights = list(self.profile.values())
        return rng.choices(classes, weights=weights, k=1)[0]
    def next_seq(self) -> int:
        self.seq += 1
        return self.seq
def _make_bbox(cls: str, rng: random.Random) -> list[int]:
    """生成该类别尺度合理的检测框。"""
    visual = CLASS_VISUAL[cls]
    lo, hi = visual["size"]
    w = rng.randint(lo, hi)
    h = int(w * rng.uniform(0.6, 1.2))
    x1 = rng.randint(0, max(1, FRAME_W - w - 1))
    y1 = rng.randint(0, max(1, FRAME_H - h - 1))
    return [x1, y1, x1 + w, y1 + h]
def _make_confidence(cls: str, rng: random.Random) -> float:
    lo, hi = CLASS_VISUAL[cls]["conf"]
    return round(rng.uniform(lo, hi), 3)
def generate_raw_frame(device: SimDevice, rng: random.Random) -> list[dict[str, Any]]:
    """生成一帧的原始检测结果（未经时序校验）。
    这里会故意混入「瞬态噪声」—— 模拟浪花反光、飞鸟、光斑。
    真实边缘盒的原始输出就长这样：大部分是噪声，靠时序校验筛。
    两种检测的区别体现在连续性上：
    - 持续目标：位于少量固定区域（每台设备维护若干个"垃圾位置"），
      逐帧微小漂移，因此能被跟踪器连续命中
    - 瞬态噪声：每次随机落点，前后帧对不上，被跟踪器自然饿死
    """
    dets: list[dict[str, Any]] = []
    # 持续目标（会被时序校验确认）
    # 每台设备有 2~4 个固定的「垃圾聚集位置」，模拟水面漂浮物
    if not hasattr(device, "_litter_spots"):
        device._litter_spots = [  # type: ignore[attr-defined]
            (
                rng.randint(100, FRAME_W - 300),
                rng.randint(80, FRAME_H - 200),
                device.pick_class(rng),
            )
            for _ in range(rng.randint(2, 4))
        ]
    for spot_x, spot_y, cls in device._litter_spots:  # type: ignore[attr-defined]
        if rng.random() < 0.78:      # 每帧较大概率出现 → 能连续命中
            w = rng.randint(30, 90)
            h = int(w * rng.uniform(0.7, 1.1))
            # 逐帧微小漂移（水流推动），位移 < match_distance 才能被匹配上
            jitter_x = rng.randint(-14, 14)
```

```
            jitter_y = rng.randint(-10, 10)
            x1 = max(0, min(FRAME_W - w - 1, spot_x + jitter_x))
            y1 = max(0, min(FRAME_H - h - 1, spot_y + jitter_y))
            dets.append(
                {
                    "class": cls,
                    "confidence": _make_confidence(cls, rng),
                    "bbox": [x1, y1, x1 + w, y1 + h],
                }
            )
    # 瞬态噪声（不会通过时序校验，被跟踪器饿死）
    if rng.random() < 0.50:
        for _ in range(rng.randint(1, 3)):
            cls = rng.choice(WASTE_CLASSES)
            dets.append(
                {
                    "class": cls,
                    # 噪声置信度偏低，进一步降低通过率
                    "confidence": round(rng.uniform(0.30, 0.62), 3),
                    "bbox": _make_bbox(cls, rng),
                }
            )
    return dets
# ---------------------------------------------------------------------
# 模拟器主体
# ---------------------------------------------------------------------
class EdgeSimulator:
    """边缘盒模拟器。"""
    def __init__(self, args: argparse.Namespace, config: dict[str, Any]) -> None:
        self.args = args
        self.config = config
        self.rng = random.Random(args.seed)
        self.site_id = config.get("site_id", "lianjiang")
        self.prefix = "marine"
        self.mqtt_conf = config.get("mqtt", {})
        self.temporal_conf = config.get("temporal", {})
        self.reporting_conf = config.get("reporting", {})
        self.buffer_conf = config.get("buffer", {})
        self.inference_conf = config.get("inference", {})
        self.devices = self._load_devices()
        self.buffer = OutboxBuffer(max_size=int(self.buffer_conf.get("max_size", 1000)))
        # 统计
        self.counter = {
            "raw_frames": 0,
            "raw_detections": 0,
            "confirmed_events": 0,
            "suppressed_noise": 0,
            "published": 0,
            "duplicates": 0,
            "buffered": 0,
```

```
            "flushed": 0,
            "net_drops": 0,
        }
        self._stop = False
        self._connected = False
        self._client: mqtt.Client | None = None
    # 初始化
    def _load_devices(self) -> list[SimDevice]:
        devices: list[SimDevice] = []
        wanted = None
        if self.args.devices:
            wanted = {d.strip() for d in self.args.devices.split(",") if d.strip()}
        for raw in self.config.get("devices", []):
            if wanted and raw["device_id"] not in wanted:
                continue
            profile = raw.get("profile", {})
            if not profile:
                # 未配置构成时给个通用分布
                profile = {c: 0.25 for c in WASTE_CLASSES}
            dev = SimDevice(
                device_id=raw["device_id"],
                name=raw.get("name", raw["device_id"]),
                device_type=raw.get("device_type", "shore_camera"),
                lng=float(raw["lng"]),
                lat=float(raw["lat"]),
                profile=profile,
                rate=float(raw.get("rate", 1.0)),
                patrol_radius_deg=float(raw.get("patrol_radius_deg", 0.0)),
            )
            if self.temporal_conf.get("enabled", True):
                dev.validator = TemporalValidator(
                    window_frames=int(self.temporal_conf.get("window_frames", 15)),
                    min_hits=int(self.temporal_conf.get("min_hits", 3)),
                    grid_size=int(self.temporal_conf.get("grid_size", 64)),
                    min_confidence=float(self.temporal_conf.get("min_confidence", 0.45)),
                    # 下面三个必须显式传：漏传会静默落到类默认值，
                    #   于是 config.yaml 里改了也不生效：排查起来极难。
                    match_distance=float(self.temporal_conf.get("match_distance", 60.0)),
                    max_misses=int(self.temporal_conf.get("max_misses", 5)),
                    min_iou=float(self.temporal_conf.get("min_iou", 0.25)),
                    decimation=int(self.temporal_conf.get("decimation", 4)),
                )
            devices.append(dev)
        # 恢复 seq（崩溃重启后从上次值续上，保证幂等判断有效）
        state = self._load_state()
        for dev in devices:
            if dev.device_id in state:
                dev.seq = int(state[dev.device_id])
        return devices
    def _state_path(self) -> Path:
```

```
        return Path(__file__).with_name(STREAM_STATE_FILE)



    def _load_state(self) -> dict[str, int]:
        path = self._state_path()
        if not path.exists():
            return {}
        try:
            return json.loads(path.read_text(encoding="utf-8"))
        except Exception:   # noqa: BLE001
            return {}
    def _save_state(self) -> None:
        try:
            data = {d.device_id: d.seq for d in self.devices}
            self._state_path().write_text(
                json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8"
            )
        except Exception:   # noqa: BLE001
            pass
    # MQTT
    def _new_client(self) -> mqtt.Client:
        """创建 MQTT 客户端。
        注意 paho-mqtt 2.x 的 callback_api_version 变更——
        传错会直接抛 ValueError，这是最常见的踩坑点。
        """
        try:
            client = mqtt.Client(
                mqtt.CallbackAPIVersion.VERSION2,
                client_id=f"edge-sim-{uuid.uuid4().hex[:8]}",
                protocol=mqtt.MQTTv5,
            )
        except (AttributeError, ValueError):
            # 回退到 1.x API
            client = mqtt.Client(client_id=f"edge-sim-{uuid.uuid4().hex[:8]}")
        if self.mqtt_conf.get("username"):
            client.username_pw_set(
                self.mqtt_conf["username"], self.mqtt_conf.get("password", "")
            )
        # 遗嘱消息：异常断开时 Broker 代发，平台据此判定离线
        if self.mqtt_conf.get("use_lwt", True):
            for dev in self.devices:
                pass  # LWT 每设备一条，见下面 _register_lwt
            self._lwt_topic = f"{self.prefix}/{self.site_id}/+/{'status'}"
        client.on_connect = self._on_connect
        client.on_disconnect = self._on_disconnect
        client.on_publish = self._on_publish
        return client
    def _on_connect(self, client, userdata, flags, reason_code, properties=None) -> None:
        self._connected = True
        print(f"[MQTT] 已连接 Broker {self.args.host}:{self.args.port}")
        # 上线后立即：1) 宣告在线 2) 补传断网期间积压的事件
        for dev in self.devices:
```

```
            self._publish_status(dev, online=True)



        if len(self.buffer) > 0:
            print(f"[补传] 恢复连接，buffer 中有 {len(self.buffer)} 条待补发")
            sent = self.flush_buffer()
            print(f"[补传] 本轮补发 {sent} 条")
    def _on_disconnect(self, client, userdata, *args) -> None:
        self._connected = False
        print("[MQTT] 连接已断开，事件将进入本地缓存")
    def _on_publish(self, *args) -> None:
        pass
    # 上报
    def _topic_event(self, dev: SimDevice) -> str:
        return f"{self.prefix}/{self.site_id}/{dev.device_id}/event"
    def _topic_telemetry(self, dev: SimDevice) -> str:
        return f"{self.prefix}/{self.site_id}/{dev.device_id}/telemetry"
    def _topic_status(self, dev: SimDevice) -> str:
        return f"{self.prefix}/{self.site_id}/{dev.device_id}/status"
    def _publish(self, topic: str, payload: dict[str, Any], qos: int) -> None:
        """发布消息。未连上或模拟断网时进本地缓存。"""
        body = json.dumps(payload, ensure_ascii=False)
        if self.args.dry_run:
            print(f"[DRY-RUN] {topic} qos={qos}\n  {body[:200]}")
            self.counter["published"] += 1
            return
        if not self._connected:
            if self.buffer_conf.get("enabled", True):
                self.buffer.put(topic, payload, qos)
                self.counter["buffered"] += 1
            return
        try:
            result = self._client.publish(topic, body, qos=qos, retain=False)
            if getattr(result, "rc", 0) != 0:
                # 发布队列满等异常 → 缓存待补
                if self.buffer_conf.get("enabled", True):
                    self.buffer.put(topic, payload, qos)
                    self.counter["buffered"] += 1
            else:
                self.counter["published"] += 1
        except Exception as exc:   # noqa: BLE001
            print(f"[警告] 发布失败: {exc}（已缓存待补传）")
            self.buffer.put(topic, payload, qos)
            self.counter["buffered"] += 1
    def _publish_status(self, dev: SimDevice, online: bool) -> None:
        """设备上下线状态 (retained) 。"""
        payload = {
            "device_id": dev.device_id,
            "online": online,
            "ts": _now_iso(),
        }
        if self.args.dry_run or self._client:
```

```
        self._publish(self._topic_status(dev), payload, qos=1)



def _publish_telemetry(self, dev: SimDevice) -> None:
    """心跳遥测。QoS0：丢一帧无所谓。"""
    payload = {
        "device_id": dev.device_id,
        "device_type": dev.device_type,
        "timestamp": _now_iso(),
        "status": "online",
        "battery": dev.battery,
        "location": {"lng": dev.lng, "lat": dev.lat},
    }
    self._publish(self._topic_telemetry(dev), payload, qos=0)
def build_event(self, dev: SimDevice, det: dict[str, Any]) -> dict[str, Any]:
    """构造符合平台契约的事件报文。"""
    seq = dev.next_seq()
    lng, lat = dev.random_location(self.rng)
    cls = det["class"]
    return {
        "event_id": f"evt_{dev.device_id}_{seq:06d}",
        "device_id": dev.device_id,
        "device_type": dev.device_type,
        "timestamp": _now_iso(),
        "location": {"lng": round(lng, 6), "lat": round(lat, 6)},
        "detections": [det],
        "aggregate": {
            "main_class": cls,
            "count": 1,
            "max_confidence": det["confidence"],
        },
        "evidence_url": None,
        "model_version": self.inference_conf.get("model_version", "det_v0.1.0"),
        "seq": seq,
    }
# 主循环
def _tick_device(self, dev: SimDevice) -> list[dict[str, Any]]:
    """跑一台设备的一帧：生成原始检测 → 时序校验 → 产出确认事件。"""
    raw = generate_raw_frame(dev, self.rng)
    self.counter["raw_frames"] += 1
    self.counter["raw_detections"] += len(raw)
    if dev.validator is None:
        # 未启用时序校验：直接放行置信度达标的检测
        threshold = float(self.inference_conf.get("confidence_threshold", 0.45))
        return [d for d in raw if d["confidence"] >= threshold]
    confirmed = dev.validator.push(raw)
    # 抽帧推理下，miss 衰减必须按「被跳过的帧」推进，不能按 push 次数
    dev.validator.sweep(dev.validator.decimation)
    self.counter["confirmed_events"] += len(confirmed)
    return confirmed
def _cooldown_ok(self, dev: SimDevice, det: dict[str, Any]) -> bool:
    """上报节流：同一格网冷却期内不重复上报。
```

没有这一步，一块持续存在的泡沫会被连续上报几十次，

平台的派单防抖虽然有 10 分钟合并窗口，但事件表会被灌爆。
--event-interval 可覆盖配置中的冷却值，用来控制演示节奏
（真实边缘盒用 90 秒；演示时想快点看到事件可设成 3~5 秒）。
"""
if self.args.event_interval is not None:
    cooldown = float(self.args.event_interval)
else:
    cooldown = float(self.reporting_conf.get("cooldown_seconds", 90))
if cooldown <= 0:
    return True
if dev.validator is not None:
    cell = dev.validator._cell(det["bbox"])
else:
    cell = (0, 0)
now = time.time()
last = dev._last_report.get(cell, 0.0)
if now - last < cooldown:
    return False
dev._last_report[cell] = now
return True
def flush_buffer(self) -> int:
"""补传本地缓存的事件。"""
batch = int(self.buffer_conf.get("flush_batch", 20))
def _send(topic: str, payload: str, qos: int) -> None:
    try:
        self._client.publish(topic, payload, qos=qos, retain=False)
    except Exception: # noqa: BLE001
        pass
sent = self.buffer.flush(_send, batch=batch)
self.counter["flushed"] += sent
return sent
def run(self) -> None:
"""启动主循环。"""
if self.args.dry_run:
    self._client = None
else:
    self._client = self._new_client()
    try:
        self._client.connect(
            self.args.host,
            self.args.port,
            keepalive=int(self.mqtt_conf.get("keepalive", 60)),
)
        self._client.loop_start()
    except Exception as exc: # noqa: BLE001
        print(f"[错误] 无法连接 MQTT Broker {self.args.host}:{self.args.port}")
        print(f"   {exc}")
        print("   提示：先执行 docker compose up -d emqx，或用 --dry-run 查看报文")
        sys.exit(2)

```
            time.sleep(1)



    self._print_banner()
    started = time.time()
    tick = 0
    # 关键：帧间隔模拟摄像头的抽帧节奏，不是「事件上报间隔」。
    #   真实边缘盒抽帧到 ~8fps（每 3 帧取 1 帧），所以帧间隔约 0.125s。
    #   时序校验的 window_frames=15 帧 ≈ 1.9 秒，正好覆盖一个浪周期。
    #   若把帧间隔设成几秒（早期实现就是这样），15 帧要跑一分钟，
    #   窗口长年填不满，用户会以为「系统不工作」。
    frame_interval = max(0.01, float(self.args.interval))
    # 事件上报冷却：真正的演示节奏旋钮。
    # None 表示用配置文件里的 cooldown_seconds（真实边缘盒的行为）。
    cooldown_override = self.args.event_interval
    try:
        while not self._stop:
            tick += 1
            elapsed = time.time() - started
            if self.args.duration and elapsed >= self.args.duration:
                print(f"\n[完成] 已达到运行时长 {self.args.duration}s")
                break
            # 模拟网络抖动
            if (
                self.args.net_drop_every
                and tick % self.args.net_drop_every == 0
                and self._connected
            ):
                self._simulate_net_drop()
            for dev in self.devices:
                confirmed = self._tick_device(dev)
                # 速率倍率 > 1 时，一帧内重复抽检（模拟更密集的垃圾分布）
                extra_rounds = max(0, int(dev.rate) - 1)
                for _ in range(extra_rounds):
                    confirmed.extend(self._tick_device(dev))
                for det in confirmed:
                    if not self._cooldown_ok(dev, det):
                        continue
                    event = self.build_event(dev, det)
                    self._emit_event(dev, event)
                # 每 5 轮发一次遥测心跳
                if tick % 5 == 0:
                    self._publish_telemetry(dev)
                    dev.battery = max(5, dev.battery - 1)
            # buffer 有积压且已恢复连接 → 持续补传
            if len(self.buffer) > 0 and self._connected and not self.args.dry_run:
                self.flush_buffer()
            self._print_progress(tick)
            self._save_state()
            # 精确对齐帧节奏：扣除本轮实际耗时，避免漂移
            sleep_for = frame_interval - (time.time() - started - elapsed)
            if sleep_for > 0:
```

```
                time.sleep(sleep_for)



    except KeyboardInterrupt:
        print("\n[中断] 收到 Ctrl+C，正在退出...")
    finally:
        self._shutdown()
def _emit_event(self, dev: SimDevice, event: dict[str, Any]) -> None:
    """发布事件，并按 --dup-rate 概率重复投递（验证平台幂等）。"""
    topic = self._topic_event(dev)
    self._publish(topic, event, qos=1)
    # 故意重复投递：模拟 QoS1 在网络抖动下的重传
    dup_rate = float(self.args.dup_rate or 0)
    if dup_rate > 0 and self.rng.random() < dup_rate:
        self.counter["duplicates"] += 1
        self._publish(topic, event, qos=1)
    label = CLASS_LABELS.get(event["aggregate"]["main_class"], "")
    print(
        f"  ↑ {dev.device_id} seq={event['seq']:>5} "
        f"{label} conf={event['aggregate']['max_confidence']:.2f} "
        f"({event['location']['lng']:.4f},{event['location']['lat']:.4f})"
    )
def _simulate_net_drop(self) -> None:
    """模拟一次网络中断：断开 → N 秒后恢复。"""
    self.counter["net_drops"] += 1
    print(f"\n[网络] 模拟断网（第 {self.counter['net_drops']} 次），事件转入本地缓存...")
    try:
        self._client.disconnect()
    except Exception:   # noqa: BLE001
        pass
    self._connected = False
    time.sleep(2.0)
    print("[网络] 尝试重连...")
    try:
        self._client.reconnect()
        time.sleep(1.0)
    except Exception as exc:   # noqa: BLE001
        print(f"[网络] 重连失败：{exc}")
def _shutdown(self) -> None:
    self._save_state()
    if self._client is not None:
        try:
            # 优雅下线：发 retained 离线状态，让平台立刻感知
            for dev in self.devices:
                self._publish_status(dev, online=False)
            time.sleep(0.3)
            self._client.loop_stop()
            self._client.disconnect()
        except Exception:   # noqa: BLE001
            pass
    self._print_summary()
# 输出
```

```
    def _print_banner(self) -> None:
        total_rate = sum(d.rate for d in self.devices)
        window = self.temporal_conf.get("window_frames", 15)
        fps = 1.0 / max(0.01, float(self.args.interval))
        cooldown = (
            float(self.args.event_interval)
            if self.args.event_interval is not None
            else float(self.reporting_conf.get("cooldown_seconds", 90))
        )
        print("=" * 66)
        print("  探海灵眸 Oceanus — 边缘感知模拟器")
        print("=" * 66)
        print(f"  站点     : {self.site_id}")
        print(f"  设备数    : {len(self.devices)} 台 (合计速率倍率 {total_rate:.1f}) ")
        print(f"  场景     : {self.args.scenario}")
        print(f"  时序校验   : {'启用' if self.temporal_conf.get('enabled', True) else '关闭'}"
              f" (窗口 {window} 帧 / 最少 {self.temporal_conf.get('min_hits', 3)} 次命中 "
              f"≈ {window / fps:.1f}s 填满) ")
        print(f"  抽帧节奏   : {self.args.interval:.3f}s/帧 (≈{fps:.1f} fps) ")
        print(f"  上报冷却   : {cooldown:.0f}s / 目标")
        print(f"  重复投递比例 : {self.args.dup_rate:.0%}")
        print(f"  断网模拟   : {'每 %d 轮一次' % self.args.net_drop_every if self.args.net_drop_e…
        if self.args.dry_run:
            print("  模式     : DRY-RUN (不连接 Broker, 只打印报文) ")
        print("=" * 66)
        print("  设备清单: ")
        for dev in self.devices:
            print(f"    · {dev.device_id:<18} {dev.name}  起始 seq={dev.seq}")
        print("=" * 66)
    def _print_progress(self, tick: int) -> None:
        if tick % 10 != 0:
            return
        c = self.counter
        # 增量累加各设备的统计 (drain 会清零, 避免重复计数)
        tracked = 0
        for dev in self.devices:
            if dev.validator is not None:
                st = dev.validator.drain_stats()
                c["suppressed_noise"] += st["suppressed"]
                c["raw_detections"] += st["fed"]
                tracked += st["tracked"]
        noise_ratio = (
            c["suppressed_noise"] / c["raw_detections"] if c["raw_detections"] else 0
        )
        print(
            f"  [统计] 轮次={tick} 原始检测={c['raw_detections']} "
            f"确认事件={c['confirmed_events']} 噪声过滤={noise_ratio:.0%} "
            f"跟踪中={tracked} 已发布={c['published']} 缓存={len(self.buffer)}"
        )
    def _print_summary(self) -> None:
```

```
        c = self.counter



        # 把各设备剩余的统计收干净，保证汇总数字完整。
        # 口径与 _print_progress 保持一致：fed / absorbed / suppressed 同单位（都是检测数）。
        tracked = 0
        for dev in self.devices:
            if dev.validator is not None:
                st = dev.validator.drain_stats()
                c["suppressed_noise"] += st["suppressed"]
                c["raw_detections"] += st["fed"]
                tracked += st["tracked"]
        print("\n" + "=" * 66)
        print("  运行统计")
        print("=" * 66)
        print(f"  原始帧数     : {c['raw_frames']}（7 台设备逐帧判定）")
        print(f"  原始检测数    : {c['raw_detections']}（含瞬态噪声）")
        print(f"  时序校验确认事件：{c['confirmed_events']}")
        print(f"  被挡下的误报   : {c['suppressed_noise']}")
        if c["raw_detections"]:
            print(f"  误报抑制率    : {c['suppressed_noise'] / c['raw_detections']:.1%}")
        print(f"  成功发布     : {c['published']}（含重复投递 {c['duplicates']}）")
        print(f"  本地缓存/补传  : {c['buffered']} / {c['flushed']}")
        print(f"  模拟断网次数   : {c['net_drops']}")
        print("=" * 66)
        print("  说明：误报抑制率 = 从未被稳定跟踪的检测数 / 喂进校验器的检测总数。")
        print("        三个计数（fed/absorbed/suppressed）同单位，因此抑制率恒在")
        print("        0~100%，且满足 fed = absorbed + suppressed。")
        print("        抑制来自两道闸：① 置信度门限直接丢弃；")
        print("        ② 抽帧推理下，只在单帧闪现的浪花几乎不会被连续命中，")
        print("           凑不满 min_hits 便被窗口自然淘汰。")
        print("        注意分母是「检测数」不是「帧数」——同一目标在连续帧里")
        print("        贡献多个检测，稳定目标因此能被反复吸收。")
        print("=" * 66)
        print("  下一步：打开 http://localhost:8000/docs 查看事件是否入库，")
        print("          或访问前端大屏看告警与工单。")
        print("=" * 66)
# ---------------------------------------------------------------------
# 场景
# ---------------------------------------------------------------------
def apply_scenario(args: argparse.Namespace, config: dict[str, Any]) -> dict[str, Any]:
    """按场景调整参数。
    注意 `--interval` 已改为「抽帧间隔」（默认 0.125s ≈ 8fps），
    场景预设不再改它：真实摄像头帧率不因演示场景而变。
    控制演示节奏的是 `--event-interval`（上报冷却）。
    """
    scenario = args.scenario
    if scenario == "stress":
        # 压测：放宽确认门槛 + 缩短冷却 + 提高设备速率，把派单引擎压到极限
        config["temporal"]["window_frames"] = 6
        config["temporal"]["min_hits"] = 2
        config["reporting"]["cooldown_seconds"] = 5
```

```
        config["buffer"]["max_size"] = 5000




        if args.event_interval is None:
            args.event_interval = 1.0
        for dev in config.get("devices", []):
            dev["rate"] = float(dev.get("rate", 1.0)) * 3
    elif scenario == "regression":
        # 回归：时序校验更严格、冷却更长，结果稳定可复现，用于冒烟比对
        config["temporal"]["window_frames"] = 20
        config["temporal"]["min_hits"] = 5
        config["reporting"]["cooldown_seconds"] = 180
        if args.event_interval is None:
            args.event_interval = 10.0
    elif scenario == "demo":
        # 演示：真实帧率 + 配置文件里的冷却（90s）。
        # 想让事件出得更快，显式传 --event-interval 3。
        pass
    else:
        print(f"[警告] 未知场景 {scenario}，按 demo 处理")
    return config
# ---------------------------------------------------------------------
# 工具
# ---------------------------------------------------------------------
def _now_iso() -> str:
    """带本地时区偏移的 ISO8601 时间戳。"""
    return datetime.now(timezone.utc).astimezone().isoformat(timespec="seconds")
def load_config(path: Path) -> dict[str, Any]:
    if not path.exists():
        print(f"[错误] 配置文件不存在：{path}", file=sys.stderr)
        sys.exit(1)
    with path.open("r", encoding="utf-8") as fh:
        return yaml.safe_load(fh) or {}
def parse_args(argv: Iterable[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="探海灵眸 Oceanus — 边缘感知模拟器",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "示例：\n"
            "  python simulator.py --scenario demo --event-interval 3 --loop\n"
            "      # 演示：8fps 抽帧 + 3 秒冷却，最快看到事件\n"
            "  python simulator.py --scenario stress --event-interval 1\n"
            "      # 压测：放宽确认门槛 + 1 秒冷却，压派单引擎\n"
            "  python simulator.py --scenario regression --seed 42 --duration 60\n"
            "      # 回归：固定种子，结果可复现\n"
            "  python simulator.py --dry-run --duration 10\n"
            "      # 只看报文，不连 Broker\n"
            "  python simulator.py --devices CAM-MABI-01 --dup-rate 0.3\n"
            "      # 验证平台幂等判重\n"
        ),
    )
    parser.add_argument("--config", default=str(DEFAULT_CONFIG), help="配置文件路径")
```

```
    parser.add_argument(
        "--scenario", default="demo", choices=["demo", "stress", "regression"], help="运行场景"
    )
    parser.add_argument("--host", default=os.getenv("MQTT_HOST", "localhost"), help="MQTT Broker 地…
    parser.add_argument("--port", type=int, default=int(os.getenv("MQTT_PORT", "1883")), help="MQTT…
    parser.add_argument(
        "--interval",
        type=float,
        default=0.125,
        help="抽帧间隔秒数（模拟摄像头帧率），默认 0.125 = 8fps",
    )
    parser.add_argument(
        "--event-interval",
        type=float,
        default=None,
        help="同一目标的最小上报间隔秒数（覆盖配置里的 90s，用于加快演示节奏）",
    )
    parser.add_argument("--loop", action="store_true", help="循环运行不退出")
    parser.add_argument("--duration", type=int, default=0, help="运行时长（秒），0=不限")
    parser.add_argument("--seed", type=int, default=None, help="随机种子（可复现）")
    parser.add_argument("--devices", default=None, help="只模拟指定设备，逗号分隔")
    parser.add_argument("--dup-rate", type=float, default=0.0, help="重复投递比例 0~1")
    parser.add_argument("--net-drop-every", type=int, default=0, help="每 N 轮模拟一次断网")
    parser.add_argument("--dry-run", action="store_true", help="只打印不发送")
    return parser.parse_args(argv)
def main(argv: Iterable[str] | None = None) -> int:
    args = parse_args(argv)
    _require_deps()          # 只在真正运行时检查依赖，便于纯逻辑单测
    config = load_config(Path(args.config))
    config = apply_scenario(args, config)
    # 命令行显式传入的 Broker 地址覆盖配置文件
    if args.host != "localhost":
        config.setdefault("mqtt", {})["host"] = args.host
    if args.port != 1883:
        config.setdefault("mqtt", {})["port"] = args.port
    # --loop 语义：不设 --duration 时就是无限循环（run() 内部靠 _stop 退出）；
    #          设了 --duration 则跑完一轮后重新开始，直到 Ctrl+C。
    if args.loop:
        args.duration = 0
    sim = EdgeSimulator(args, config)
    def _sigterm(_signum, _frame):
        sim._stop = True
    try:
        signal.signal(signal.SIGTERM, _sigterm)
    except (ValueError, OSError):
        pass   # 非主线程环境（如测试）无法注册信号
    sim.run()
    return 0
if __name__ == "__main__":
    sys.exit(main())
```